COMPLETE ECOSYSTEM ARCHITECTURE

How Inaya Actually Works

Every contract, app, backend, and data flow across the ecosystem — written from the real code, not the pitch. For the founding team.

Document INAYA-ARCH-2026-V1 · Classification Internal · October 2026

SECTION 01

How To Read This Document

Inaya is one product with two connected engines: a decentralized storage/DePIN protocol (on-chain contracts, node operators, client-side encryption) and a set of applications built on top of it (the web dApp, Business Workspace SaaS, mobile app, two desktop apps, and three AI assistants). This document maps every piece, what it actually does, and how it connects to everything else.

Conventions used throughout:

- Everything here is on BNB Chain Testnet unless stated otherwise — nothing described is live on mainnet yet.
- Contract addresses given are real, deployed, and verified on BscScan as of this writing.
- Where a feature is deliberately unbuilt or deferred, it's labeled as such rather than glossed over — this is a working reference, not a pitch document.
- "Backend" always means the Next.js app at inaya-network-dapp — every mobile screen, desktop app, and the web dApp itself all call the same backend, there is no separate API server.

SECTION 02

The System At A Glance

REPO / APP	WHAT IT IS	RUNS ON
inaya-network-dapp	The backend (Next.js API routes + MongoDB) AND the main web dApp AND the Business Workspace web UI — one Next.js app serving all three.	Vercel, Node.js
contracts/ (repo root)	Solidity source for the protocol's own contracts (Staking, Vault, Escrow, Node Registry, Proof Registry, Security Layer). Deployed to BSC Testnet.	Solidity 0.8.20, Hardhat
custody-sdk	Client-side encryption/sharding library ("InayaKernel") — the code that actually encrypts and splits a file before it ever leaves a user's device.	JS, @noble crypto primitives, ethers v6
custody-sdk/packages/node-daemon	CLI a storage-node operator runs — registers on-chain, sends heartbeat telemetry. Published to npm.	Node.js CLI
inaya-mobile	React Native / Expo mobile app — a superset of the web dApp's consumer features plus Business Workspace, Learn, and Security.	Expo, React Native
inaya-desktop	Tauri (Rust) native wrapper around the Business Workspace web app.	Tauri, WebView2 (Win) / WebKitGTK (Linux)
inaya-dapp-desktop	Tauri native wrapper around the main dApp (faucet, staking, etc).	Tauri, same engines as above

> None of the desktop or mobile apps run their own backend logic beyond thin native chrome — every one of them calls the exact same inaya-network-dapp API routes the website does.

SECTION 03

The Two Engines

ENGINE 1 — DEPIN PROTOCOL

The foundation: users encrypt files client-side, shard them, and anchor ownership + integrity proofs on-chain. Independent node operators register capacity, earn $INAYA/USDT for storage and threat-reporting, and settle through timelocked, oracle-attested (not cryptographic) verification. This is the part that makes Inaya a real DePIN, not just a SaaS product with a token bolted on.

ENGINE 2 — APPLICATIONS

Everything a user actually touches: the web dApp, Business Workspace (SaaS document management for companies that will never think about blockchain), mobile app, two desktop wrappers, a public Security transparency page, an investor Data Room, and three Gemini-powered AI assistants. All of it sits on top of Engine 1's infrastructure, but most of it is reachable — and usable — without a wallet.

> **Why this split matters.** It's the reason a non-crypto business customer can use the Business Workspace with just an email address while a node operator, three layers down, is settling USDT through a timelocked escrow contract — the same infrastructure serves both, but almost nothing about Engine 1 leaks into Engine 2's user experience.

SECTION 04

On-Chain Layer — Contracts

10 contracts deployed and verified on BSC Testnet, split across two families that don't share any direct dependency on each other.

Core Protocol (6 contracts)

Custody, payments, staking, and node settlement.

CONTRACT	ADDRESS	JOB
InayaCustody ("Core Custody Contract")	0x7F5E6c...5a888	Asset-ownership ledger. batchRegisterAssets() writes fileHash → {owner, shard CIDs, size, timestamp}. This is the single source of truth for "who owns this file." Note: its Solidity source isn't in this repo — only its interface (IInayaCustody) is, referenced from InayaProofRegistry. It's deployed and live, just not tracked as source here yet.
Mock USDT (mUSDT)	0x6f16E2...45D	Testnet stand-in for USDT — every fee, stake, escrow, and invoice in the protocol is denominated in this token until mainnet.
$INAYA Token	0x3966a3...e94e	The real ERC-20 utility/governance token. Fixed supply, 30,000,000 tokens (per tokenomics).
InayaNodeRegistry	0xd12a38...FD881	Node operator registry + timelocked commission settlement. registerNode(capacityGB), tiered commission (30/40/50% by Entry/Mid/Enterprise), verifier-attested metrics, 36-hour settlement delay before anyone can call releaseSettlement().
RevenueRouter	0x76B0d4...3256	processCorporateInvoice(usdtAmount) — entry point for Corporate Reserve plan purchases. Source not in this repo, referenced by address from the Stripe webhook and page.js.
InayaProofRegistry	0xEdF431...CEBcB	Merkle-root storage per asset + chunk-proof verification (registerMerkleRoot, verifyChunkProof). Reads InayaCustody to confirm the caller owns the asset before letting them register a root.

> Also in contracts/ but not part of the six above: InayaStaking (Synthetix-style staking-rewards pool, 0/30/90-day lock tiers at 1.00x/1.25x/1.50x multiplier), InayaEgressTimelockVault (holds egress-fee $INAYA in 180-day epochs, forwards it to Staking; also sweeps USDT storage revenue to the operational treasury), InayaCorporateEscrow (drips a node operator's 39% COGS share of a corporate invoice over 12 monthly releases), and MockINAYA (test-only ERC-20 stand-in for $INAYA, not deployed to testnet under the real token address). All four are deployed via their own scripts/deploy-*.cjs; addresses live in .env.local and are read by page.js as stakingContractAddress / etc. — not restated here to avoid publishing addresses that weren't independently re-verified for this document.

Security Layer (4 contracts)

Decentralized threat intelligence — fully separate dependency graph from the core protocol above. Full detail in Section 09.

CONTRACT	ADDRESS	JOB
InayaThreatRegistry	0xb374dE...8adD	Threat status ledger, updated only by InayaThreatReporter.
InayaThreatReporter	0xe22EbA...6450B	confirmThreat() — relayer-only, called once a threat crosses the confidence threshold off-chain. Writes into ThreatRegistry.
InayaNodeReputation	0xD25836...9E27A	Periodic on-chain checkpoints of node reputation scores (real-time reputation itself lives off-chain).

CONTRACT	ADDRESS	JOB
InayaSecurityPolicy	0xCE1646...1976a	Versioned policy hash + URI, so a client can verify a cached policy offline without a live RPC call.

Trust-model honesty, on the record. InayaNodeRegistry's own header comment is explicit: node metrics are "coordinator-verified," not cryptographically proven — an authorized verifier wallet attests uptime/capacity off-chain. InayaProofRegistry's chunk-proof verification is currently onlyOwner (the backend checks proofs itself); the contract's own comment lays out a documented future path to make this permissionless and stake-slashing-backed. Neither is hidden — both are stated in the contract source itself.

SECTION 05

Client-Side Encryption & Storage — the custody-sdk

"InayaKernel" (custody-sdk) is the code that actually makes the zero-knowledge claim true. Published as @inaya-network/custody-sdk, used identically by the web dApp and the mobile app.

1. **Key derivation.** deriveVaultKey() runs PBKDF2 (default SHA-256, 100,000 iterations) over the user's passkey plus a fresh random 16-byte salt, producing a 32-byte AES key. The passkey itself never leaves the device and is never transmitted anywhere.
2. **Encrypt + shard.** disperseAndSlice() reads the file, base64-encodes it, AES-GCM-256 encrypts it with a fresh random IV, prepends salt+IV to the ciphertext, then splits that resulting base64 string at its exact character midpoint into two shards (shardAlpha / shardBeta). This is literal binary bisection — neither half means anything on its own, but it is not erasure coding or Reed-Solomon.
3. **Pin to IPFS.** The SDK itself does NOT upload anywhere — that's the calling app's job. page.js pins each shard to Pinata separately and gets back two CIDs (cidAlpha, cidBeta).
4. **Anchor on-chain.** anchorToLedger() calls InayaCustody.batchRegisterAssets(fileHashes, fileSizes, shardACIDs, shardBCIDs) — the CIDs (pointers, not the shard bytes themselves) are what actually gets written on-chain.
5. **Reconstruct + decrypt.** retrieveAndReconstruct() reads InayaCustody.assets(fileHash) to get both CIDs, fetches both shards from the Pinata gateway in parallel, concatenates them back into the original base64 string, splits out salt/IV/ciphertext, re-derives the AES key from the user's passkey, and decrypts.

> **Zero-knowledge, verified against the actual code — not assumed.** Every encryption, key-derivation, and decryption step happens locally using only inputs the device already has (passkey, file bytes). No network call anywhere in custody-sdk's source carries plaintext or the derived key. The only backend/chain-touching write is a file hash, size, and two CID strings — never file content, never the key.

A second, separate feature — sharing without exchanging the raw passkey:

- encryptForPublicKey / decryptWithSecretKey / deriveEncryptionKeypairFromSignature — re-wraps the passkey itself (not the file) using X25519 ECDH + HKDF-SHA256 + XChaCha20-Poly1305 sealed-box encryption, so a recipient can unlock a shared vault entry without the passkey ever being transmitted in the clear.

> Known gap worth flagging: the SDK's own bundled example (examples/node-script.mjs) skips the Pinata step and passes raw ciphertext directly into anchorToLedger's CID parameters — it only "works" because the example never calls retrieveAndReconstruct against real IPFS. The production code path in page.js does the Pinata step correctly; the example just isn't a faithful end-to-end demo.

Trust Fabric — ZK compliance proofs, TEE attestation, intent-based routing (September 2026).

Three real, tested modules added to custody-sdk: ComplianceProofs.proveAuthorizedAccess() produces a genuine Schnorr/Fiat-Shamir zero-knowledge proof that the caller controls a key authorized for a given {policy, action, timestamp}, without revealing the key itself — production-ready, wired into InayaKernel. Intent.* is a validate/discover/simulate/risk-check/execute pipeline in front of the real bridge SDK, so a cross-chain transfer is one auditable unit rather than a raw bridge call (a route-substitution vulnerability was found and fixed during adversarial testing). Attestation.* verifies device/environment evidence against a policy — a fully-tested software-simulated TEE pipeline today, honestly labeled non-hardware-backed, not claimed as real silicon attestation. 91/91 tests passing (55 new this SOW). Full detail: custody-sdk/docs/trust-fabric-phase0-1.md and trust-fabric-phase3-adr.md.

Release verification — the SDK doesn't have to be trusted blindly.

Verifiable Inaya Client SOW (September 2026): as of custody-sdk v1.0.10-beta, every tagged release is independently reproducible and verifiable, not just claimed open-source. Full mechanism and third-party verification commands: custody-sdk/docs/VERIFYING_RELEASES.md.

1. **Reproducible build.** custody-sdk ships with no build step — src/*.js is exactly what's in git (ESM, explicit .js import extensions, hand-written .d.ts). "Reproducible" reduces to "the published tarball is provably the tagged source, file for file," not a compiled-artifact-determinism problem.

2. **Published hash — and a real bug found and fixed computing it.** Every tagged release (.github/workflows/release.yml) publishes a git-tree-hash and an npm-tarball SHA-256 to CHECKSUMS.md and the GitHub Release. First implementation used git archive HEAD | sha256sum — verification then found this isn't reproducible across git versions (confirmed directly: local git 2.55.0 and ubuntu-latest's git produced different hashes for the identical tagged v1.0.9-beta commit, which would make an honest verifier see a false mismatch). Fixed in v1.0.10-beta by publishing git rev-parse <tag>^{tree} instead — git's own content-addressed tree object id, identical on every git install by construction.

3. **Independent verification, actually performed.** Not just documented — run: v1.0.10-beta's git-tree-hash was independently recomputed on a separate machine and matched CHECKSUMS.md; the GitHub Release tarball was downloaded fresh and its SHA-256 matched; every file inside it was diffed byte-for-byte against the tagged git source and found identical. npm provenance (npm publish --provenance) separately attaches a signed, publicly-checkable statement of exactly which CI run produced the package (npm audit signatures).

4. **Content-addressed delivery.** The release tarball is also pinned to IPFS (Pinata) at publish time; the resulting directory CID is recorded in CHECKSUMS.md and the GitHub Release, IPFS pinning kept non-blocking (continue-on-error) so a Pinata-side outage can't prevent npm publish. Fetching by CID — not from any server Inaya controls — and hashing what comes back is a second, independent confirmation of the exact artifact.

> **What this does and does not prove.** Guarantees: the code that produced a release is exactly the tagged source; anyone can independently re-derive the published hashes; as of the same release, the web dApp (src/lib/clientCrypto.js, src/app/page.js) and inaya-mobile both call this published npm package directly rather than each running its own separate crypto — verified by a committed cross-implementation test, custody-sdk/test/webCryptoCompat.test.mjs, proving the SDK's @noble-based AES-GCM/PBKDF2 is byte-identical to and cross-decryptable with the dApp's former inline crypto.subtle implementation. Does not guarantee: that inayanetwork.com is serving this exact build at this exact moment, or an implementation-bug-free audit of the primitives beyond that cross-compatibility test.

SECTION 06

Node Operator Layer

What someone actually runs to operate an Inaya storage node today, and what it does and does not do.

NODE-DAEMON (PUBLISHED NPM CLI)

Commands: login, register <capacityGB>, start, report <indicator> (Security Layer), service install/uninstall. Wallet key is PBKDF2+AES-GCM encrypted at rest locally (~/.inaya/node-daemon/config.json). register() calls InayaNodeRegistry.registerNode() on-chain AND separately POSTs to the backend's /api/nodes/register for capacity bookkeeping. start() runs a 5-minute heartbeat loop POSTing telemetry to /api/nodes/heartbeat.

WHAT IT DELIBERATELY DOES NOT DO

It does not store or serve shards, does not host any file content, and does not execute settlement/payout logic — the daemon's own code comments are explicit that commission settlement is verifier- and relayer-driven, not something the daemon touches. Its on-chain surface is exactly two calls: registerNode and a read of nodes(address). It is an identity + registration + heartbeat agent, nothing more, today.

Settlement flow (InayaNodeRegistry, off-daemon):

- An authorized verifier wallet calls queueSettlement / queueSettlementsBatch — computes commission by tier, does not move funds yet.
- After a mandatory 36-hour delay, releaseSettlement is publicly callable by anyone (deliberately — a single compromised verifier key can queue a bad settlement but can never immediately drain funds).
- Corporate invoice revenue flows separately: RevenueRouter.processCorporateInvoice() → InayaCorporateEscrow.createEscrow() escrows a node operator's 39% COGS share and drips it out over 12 monthly releases via releaseMonthlyPayout(), callable by anyone once due.

SECTION 07

Web dApp — Protocol Surface

src/app/page.js — one large tabbed SPA. Every tab below is wallet-driven except Referrals (email + KYC only, no wallet).

TAB	WHAT IT DOES
Network Home	Landing panel only — status tiles, CTA into Sovereign Vault, cross-promo into Business Workspace / desktop downloads. No contract or API calls of its own.
Faucet	POSTs to /api/faucet; the route dispenses test $INAYA/USDT server-side from a custodial faucet wallet. Self-limits by checking the wallet's existing balance rather than a visible cooldown timer.
Sovereign Vault	The core product. Upload: encrypt+shard client-side (custody-sdk) → pin to Pinata → InayaCustody.batchRegisterAssets() → InayaProofRegistry.registerMerkleRoot() per file. Download: reverse of the same path. Also has a no-wallet path for Stripe card customers (upload happens locally, on-chain registration happens server-side post-payment).
Corporate Reserve (sidebar panel + Business Model tab)	The sidebar tier selector (250/500/1000 TB) only sets display state and enforces per-plan upload size limits — it is NOT the purchase flow. The real purchase lives on the Business Model tab: approve USDT → RevenueRouter.processCorporateInvoice() → approve USDT → InayaCorporateEscrow.createEscrow() escrowing the 39% operator COGS share over 12 months.
Staking	Reads InayaStaking for APY/tier display. Stake (with 0/30/90-day lock tier selection, real on-chain lockExpiry), Unstake (blocked until lock expires), Claim reward — direct calls to stake()/withdraw()/claimReward().
My Dashboard	Read-only aggregation of state already fetched elsewhere — PAYG transaction history with BscScan links, Corporate Reserve status, staking overview. No new contract calls.
Referrals (ReferralSection.js)	Deliberately no wallet anywhere — email + Didit KYC only, specifically to detect self-referral. Real KYC gate: referral credit only counts once Didit verification resolves.
Genesis Airdrop	Two mechanics, neither with an on-chain claim function today: an "Upload Reward" progress-bar estimate (0.01 $INAYA per upload, capped at 0.3/wallet, display-only, not minted), and a "Contributor Allocation" (Community/Developer/Moderator buckets of a 1,000,000-token pool) that routes to Google Forms for manual review — real distribution is off-app at TGE, not automated by this code.

SECTION 08

Business Workspace — SaaS Layer

A separate, structurally distinct product built on the same infrastructure: companies get organizations, departments, projects, and documents with real server-enforced permissions — sold and priced independently of storage/token economics, no wallet literacy required.

- Auth: email magic-link OR Google Sign-In, session cookie based (org.js — createSession/consumeLoginToken), completely separate from the wallet-signed-message pattern used elsewhere.
- Hierarchy: Org → Department → Project → Document, each level with its own role/permission scoping (canManageOrg gates admin actions; VIEW/EDIT/MANAGE per document).
- Workflow: documents move through a real state machine (DRAFT → PENDING → etc.) with an atomic transition function and a persisted activity log — not just a status field that gets overwritten.
- Sharing: explicit per-person grants (document-permissions.js) plus tokenized public share links (hash-and-compare, atomic single-consume where applicable) — same pattern conventions as the Data Room's magic-link/session tokens, implemented independently for each feature.
- Storage model: documents here are encrypted client-side before upload using the same passkey-based flow as the Sovereign Vault, then pinned as JSON-shard blobs — NOT the same storage path as the Data Room, which stores plain, unencrypted PDFs for inline viewing (see Section 11 for why that's a deliberate, different choice).
- Billing: Stripe subscription plans (orgPlans.js), seat limits enforced server-side on invite, storage-size limits enforced server-side on document upload.
- AI: a permission-aware Business Assistant (Section 12) that only ever sees data the requesting user is actually authorized to see.
- Beyond documents: the workspace now covers real business operations on the same org/department/permission foundation — Tasks, CRM, Procurement, Inventory (Section 23), and a Finance & HR layer (Section 24, testnet demonstration scope). None of this required touching the base document/permission model; every module is additive.

SECTION 09

Security Layer — Decentralized Threat Intelligence

A separate DePIN-style subsystem: independent nodes report suspicious domains/IPs; the backend reputation-weights reporters and, once confidence crosses a threshold, anchors a CONFIRMED verdict on-chain. Marketed externally as "Inaya Firewall."

1. **Report.** A node (mobile/desktop client or node-daemon's report command) POSTs a signed observation to /api/security/report. Rate-limited (200/day/node), upserted per node+threat+day.
2. **Aggregate.** computeThreatConfidence() reputation-weights every independent reporter over a 14-day window. Node reputation is real-time off-chain, only checkpointed on-chain periodically — confirmed reports raise it, false positives cost 3x as much (asymmetric penalty by design).
3. **Confirm.** Crossing CONFIRM_THRESHOLD_BPS (75%) triggers exactly one relayer-signed InayaThreatReporter.confirmThreat() call. A single new/low-reputation node can never force a confirmation alone — verified live in testing: 4 neutral-reputation nodes capped out at 52% confidence, well short of threshold.
4. **Surface.** Public /security web page (flagship AI-assistant card, destination checker, live threat feed, Inaya Firewall banner), mobile SecurityScreen (Monitor/Protect/Strict modes, local allow/blocklist, network overview), desktop enforcement (Windows netsh advfirewall / Linux iptables block, IP-literal only — not run against a real machine yet, reviewed line-by-line only), admin dashboard, and a dedicated Security AI Assistant that is explicitly forbidden from inventing evidence — every answer is grounded only in verified security_* collection data.

> Plaintext indicators (domains/IPs) never touch the chain — threat IDs are keccak256 of the normalized indicator; the backend holds the plaintext↔hash mapping (src/lib/security.js).

SECTION 10

Inaya Learn — Educational Platform

A YouTube-based educational discovery layer inside both the web dApp and mobile app — turns Inaya into a daily-use destination beyond storage/staking.

- Search/browse via the real YouTube Data API v3, backend-cached (MongoDB TTL index) specifically because search.list costs 100 quota units/call against a 10,000/day default quota — caching is load-bearing infrastructure here, not an optimization.
- Categories and curated collections (Learn Web3 / Learn AI / Learn Programming) are a hardcoded, git-deployed config file, not an admin-editable database — deliberate V1 scope cut.
- Local-first save/progress (AsyncStorage on mobile, localStorage on web), optional backend sync once a wallet connects.
- An AI Tutor grounded in the opposite guardrail philosophy from the Security Assistant: it teaches using its own general knowledge, only calling tools to check the user's own saved videos/progress — never gated to a narrow data set.

SECTION 11

Investor Data Room

A dedicated, branded document room for sharing investor materials with per-visitor view tracking — replaces sharing a Google Drive folder blind.

- Single shareable link, email required (not optional) + NDA click-through, per-visitor magic-link email verification.
- Deliberately different storage model from the Business Workspace: documents here are plain, unencrypted PDFs (server-proxied streaming, never a raw Pinata URL exposed) so they can be viewed inline with real duration tracking — the Business Workspace's client-side-encrypted-blob model is the wrong fit when the entire point is casual inline viewing with analytics.
- View events (open + 15-second heartbeat + close) roll up into a per-visitor engagement table for the admin — exactly who looked at what, for how long.

SECTION 12

The AI Assistants

Four assistants — Docs, Business, Security, and Learn Tutor — sharing one architectural pattern (Gemini function-calling, 5-round tool loop, 429/503 retry with backoff) but deliberately different guardrail philosophies. Three of the four (Docs, Security, Learn) are now also grounded by a shared RAG retrieval layer — see Section 25 for how that actually works; this section stays focused on what each assistant is FOR.

ASSISTANT	WHERE	GUARDRAIL PHILOSOPHY
Docs Assistant	Public site-wide chat widget	Grounded in the project's own docs/FAQ/fundraising-doc corpus via RAG retrieval (Section 25) instead of a static hardcoded knowledge block — answers cite which source they came from.
Business Assistant	Business Workspace sidebar	Permission-aware — every tool call re-checks the requesting user's actual document/project/finance/HR access; never surfaces data they couldn't otherwise see. Uses direct permission-scoped MongoDB tool-calling, not RAG — this is live, per-org private data, never appropriate to put in a shared vector index.
Security Assistant	Public /security page + mobile SecurityScreen	Strict evidence-grounding — explicitly forbidden from inventing or generalizing beyond verified security_* collection data (live tool-calling) or the RAG-indexed security policy/explainer docs (Section 25). "Explain phishing in general" is fine; fabricating a specific threat's status is not.
Learn AI Tutor	Web + mobile Learn section	The opposite on purpose — teaches using its own general knowledge like a real tutor; tools (including RAG search over the current video's transcript, Section 25) exist only to ground it in the user's own saved videos/progress, never to gate what subject matter it can discuss.

SECTION 13

Mobile App

inaya-mobile (Expo/React Native) is a superset of the web dApp's consumer features, plus Business Workspace, Learn, and Security — not a stripped-down companion app.

- Wallet: MetaMask Connect Multichain (direct deeplink to MetaMask Mobile), configured for BNB Chain Testnet specifically.
- Business Workspace: full org/department/project/document hierarchy, permissions, sharing, Bearer-token session auth (separate from web's cookie session), biometric app-lock, Google Sign-In.
- Security: SecurityScreen with protection mode, destination checker, local allow/blocklist, network overview, AI Security Assistant chat.
- Learn: search/browse/watch/save/progress via react-native-youtube-iframe, plus a global AiTutorButton (header icon → full-screen modal) reachable from every Learn screen.
- Watcher Pioneer, Referrals, Faucet-adjacent flows, and Genesis Airdrop all mirror the web dApp's mechanics 1:1 against the same backend routes.

SECTION 14

Desktop Apps

Two separate Tauri (Rust) apps — deliberately thin wrappers, no bundled frontend. Each app's dist/ is a blank placeholder HTML page; the Rust shell just points its webview at the live production URL and layers native OS chrome (tray, menu, notifications, auto-update) on top.

APP	WRAPS	NATIVE ADDITIONS BEYOND THE WEB PAGE
inaya-desktop ("Business Workspace")	inayanetwork.com/business	System tray/minimize-to-tray, native menu, native Windows toast for pending approvals (polls /api/orgs/pending-approvals), signed auto-updater, real firewall-block enforcement (netsh/iptables) for the Security Layer.
inaya-dapp-desktop ("Inaya Network")	inayanetwork.com/	Same tray/menu/updater pattern, no notifications plugin (no equivalent concept on the main dApp). Detects window.__TAURI__ and leads with WalletConnect instead of extension-based wallets, since a native webview has no browser extensions.

> Chosen over Electron after measuring both: 8.3MB installed vs 269MB for an equivalent Electron build, since Tauri rides the OS's own WebView2 (Windows) / WebKitGTK (Linux) instead of bundling Chromium. Not code-signed yet (deferred decision, see Section 20) — Windows installs show an expected SmartScreen warning.

SECTION 15

Backend Infrastructure & Conventions

One Next.js app (inaya-network-dapp) serves the web dApp, Business Workspace, every public page, and the entire API surface every client (mobile, both desktop apps) calls.

- Data layer: MongoDB, one lib file per feature domain (security.js, learn.js, dataroom.js, activity.js, orgs.js, watcherPioneer.js, feedback.js...) — each with the same shape: getXCollections(), ensureXIndexes() (module-level idempotency guard), validateXInput() functions that throw (fail-closed).
- Cron jobs (Vercel Cron, vercel.json): settlement-release (midnight), security reputation checkpoint (3am — moved off a 6-hourly schedule after it silently broke every deployment for weeks on the Hobby plan's once-daily cron limit).
- Two admin surfaces: /admin (Enterprise Dashboard — revenue, usage, DAU/WAU, feedback) and /admin/security and /admin/dataroom, all gated by the same ADMIN_DASHBOARD_SECRET ?key= pattern, not a full multi-admin auth system (solo-founder-appropriate, by design).
- Every route follows the same shape: export const dynamic = "force-dynamic", outer try/catch → 500 + console.error, inner try/catch → specific 4xx.

SECTION 16

Identity & Auth Patterns

PATTERN	USED BY	MECHANISM
Signed-message wallet auth	Sovereign Vault sign-up, Security Layer node reports, Watcher Pioneer	ethers.verifyMessage over a reconstructed message string, 5-minute freshness window — reimplemented locally per-feature rather than shared, by established convention in this codebase.
Cookie session (magic-link / Google)	Business Workspace (web)	orgs.js — generateToken/hashToken, TTL-indexed magic_links collection, consumeLoginToken.
Bearer token session	Business Workspace (mobile)	Same session concept as web, token-based instead of cookie-based since mobile has no cookie jar shared with a browser.
Social login (Google or Telegram)	Watcher Pioneer Program (added October 2026)	Google: the ID token's signature and audience are verified server-side. Telegram: a one-time link opens the program's bot, the same Telegram user confirms Yes, and the server issues a signed 30-day session token exactly once. The identity is the provider's stable account id (never the email); a wallet can be linked later, or a login attached to an existing wallet account, and two accounts are never merged. Telegram stays off until its bot is configured.
Anonymous device/visitor ID	Referrals, activity pings, Security destination checker, Learn progress (no wallet)	Client-generated UUID cached in localStorage/AsyncStorage, upgraded to wallet address once one connects — same trust model reused across every feature that needs to work pre-wallet.
Investor Data Room session	Data Room only	Its own magic-link + session-cookie pair, deliberately separate from Business Workspace's — different visitor identity model (name+email+NDA, not org membership).

SECTION 17

Activity & Analytics (DAU/WAU)

One small, shared primitive tracks daily/weekly active users across all three surfaces — dApp, Business Workspace, mobile.

- One MongoDB doc per identity+surface+day, upserted (not inserted) — repeated pings the same day are free no-ops, so dedup is structural, not query-time logic.
- DAU = distinct identities pinging today; WAU = distinct identities over the trailing 7 days — both computed with a plain distinct() query since the per-day collapse keeps the working set small.
- Identity: wallet address if connected, otherwise the same anonymous visitor/device ID pattern from Section 16.
- Surfaced as one more section on the existing /admin dashboard — not a new page, not a new auth system.

SECTION 18

End-to-End Flow: Uploading a File

1. **1.** User connects wallet, signs a verification message (Sovereign Vault sign-up), sets a master passkey, selects a file.
2. **2.** custody-sdk derives an AES key from the passkey (PBKDF2), encrypts the file (AES-GCM-256), and splits the ciphertext into two shards.
3. **3.** Each shard is pinned to Pinata/IPFS independently, returning two CIDs.
4. **4.** Client checks for a duplicate asset hash, reads live fee rates, approves USDT/$INAYA if needed, then calls InayaCustody.batchRegisterAssets() — this is the transaction that actually makes the upload "real."
5. **5.** Client separately calls InayaProofRegistry.registerMerkleRoot() per file (a Merkle root over the file's chunks) — a second, independent transaction; if this one fails, the custody registration from step 4 is not rolled back.
6. **6.** On download, the reverse: read InayaCustody.assets(fileHash) for both CIDs → fetch both shards from Pinata → concatenate → derive the same AES key from the user's passkey → decrypt. The passkey never leaves the device at any point in either direction.

SECTION 19

End-to-End Flow: A Threat Getting Confirmed

1. **1.** Independent nodes each observe the same suspicious domain/IP and POST a signed report to /api/security/report — rate-limited, deduped per node+threat+day.
2. **2.** computeThreatConfidence() reputation-weights every independent reporter within a 14-day lookback window.
3. **3.** Once aggregate confidence crosses 75%, the backend (acting as an authorized relayer) fires exactly one InayaThreatReporter.confirmThreat() call — anchoring category, confidence, and a hash of the contributing-node list on-chain.
4. **4.** The public /security page, mobile SecurityScreen, and desktop enforcement all read the same confirmed-threat feed from that point forward — a confirmed domain gets blocked at the app layer everywhere, and at the OS firewall layer on desktop for a real, confirmed threat.

SECTION 20

Known Gaps & Deliberate Deferrals

Stated plainly, not buried — these are conscious scope cuts, not things quietly forgotten:

- Node-daemon does not store/serve shards or run settlement logic — it's identity + registration + heartbeat only, today.
- InayaEgressTimelockVault has no code path yet that actually pays INAYA/USDT into it — it only accrues balance once something is wired to send it.
- InayaProofRegistry's chunk-proof verification is onlyOwner (backend-checked) rather than permissionless/stake-slashing-backed — a documented future path, not built yet.
- Real Android VPN/packet interception for the Security Layer is out of scope — mobile enforcement checks destinations inside the Inaya app only.
- Desktop firewall-block commands (netsh/iptables) are written and reviewed but have not been executed against a real machine.
- Neither desktop app is code-signed (Authenticode) — investigated, no free option exists for closed-source software, deferred to mainnet rather than spend on it now.
- Genesis Airdrop has no on-chain claim function — both mechanics are display/manual-review only, real distribution happens off-app at TGE.
- macOS is out of scope for both desktop apps, per Apple Developer Program cost + notarization constraint.
- Finance & HR (Section 24) is explicitly a testnet demonstration/validation layer — no PDF invoice generation, no multi-currency conversion, no regulated banking/payment-processor integration, no payroll/tax processing.
- RAG (Section 25) deliberately never ingests custody-sdk's own docs or the legacy KNOWLEDGE_ARTICLES collection, and never ingests private per-org Business Workspace data or per-identity Security events into the shared vector index — those stay live, permission-scoped tool calls, by design.

SECTION 21

Oracle & Automation Layer

A third, independent subsystem alongside the core protocol and Security Layer: an on-chain registry of approved external data sources, an adapter that validates every submission before trusting it, and an off-chain keeper that executes pre-approved contract actions under smart-contract rules — never arbitrary admin commands. Deployed and running live on BSC Testnet, publicly verifiable at inayanetwork.com/automation.

CONTRACT	ADDRESS	JOB
InayaOracleRegistry	0x0b4695...FdD90	Owner-approved data sources: which address may submit for which data type, active/inactive, emergency disable. Holds no data itself.
InayaOracleAdapter	0x44E9E1...4789	The actual data store other contracts read. Every submission is validated on-chain — not from the future, not already stale, not faster than the source's minimum interval, not an outlier beyond a configurable max deviation from the previous value. A submission failing any check reverts; nothing partially-invalid is ever recorded.
InayaAutomationRegistry	0xa24Eae...ADf53	A transparent record of approved automated tasks and what they've actually done — deliberately holds no special calling rights over any target contract and never forwards a call. The off-chain worker calls target functions directly, under whatever access control they already enforce.

1. **Real data, not a simulated demo.** The oracle source live today is the INAYA/USDT spot price, read directly from the live PancakeSwap testnet pool's reserves — the same price computation the egress checkout flow already uses. Not a fabricated number for demonstration purposes.
2. **Real automation target, not a toy example.** InayaNodeRegistry's releaseSettlementsBatch() was already permissionless and time-locked — anyone could call it once a settlement's 36-hour delay passed, but nothing was calling it automatically. The keeper now does, on a schedule, running against a genuine existing function rather than one built just for this demo. The first real run found and released an actual previously-unclaimed settlement.
3. **Fails safe, not silently.** If oracle data goes stale, dependent automation skips that pass rather than acting on unverified data — proven live by deliberately lowering the staleness threshold and confirming the system correctly detects and reports it, then restoring normal operation.

> The keeper is a standalone script an operator runs (manually or on their own schedule) with their own key — not a hosted service with standing infrastructure access. Its on-chain authority is narrow by construction: it can only submit to oracle sources it's explicitly registered for, and it can only call functions that are already safe to call permissionlessly.

SECTION 22

Where Everything Lives

AREA	PATH
Core protocol contracts	contracts/ (repo root) — InayaStaking.sol, InayaEgressTimelockVault.sol, InayaCorporateEscrow.sol, InayaNodeRegistry.sol, InayaProofRegistry.sol, MockINAYA.sol
Security Layer contracts	contracts/InayaThreat{Registry,Reporter}.sol, InayaNodeReputation.sol, InayaSecurityPolicy.sol
Oracle & Automation contracts	contracts/oracle/InayaOracleRegistry.sol, InayaOracleAdapter.sol, contracts/automation/InayaAutomationRegistry.sol
Deploy scripts	scripts/deploy.js, deploy-staking.cjs, deploy-vault.cjs, deploy-escrow.cjs, deploy-node-registry.cjs, deploy-verifier-safe.cjs, deploy-security-layer.js, deploy-oracle-automation.js
Automation keeper + tests	scripts/automation-worker.mjs, scripts/test-automation-worker.mjs, test/OracleAutomation.test.js
Client-side crypto SDK	inaya-network-dapp/custody-sdk/src/ (crypto.js, index.js, contracts.js)
Node operator CLI	inaya-network-dapp/custody-sdk/packages/node-daemon/
Release verification (reproducible build, published hashes, IPFS pin)	custody-sdk/.github/workflows/release.yml, CHECKSUMS.md, docs/VERIFYING_RELEASES.md, scripts/pin-release.mjs
Cross-implementation crypto compatibility proof	custody-sdk/test/webCryptoCompat.test.mjs
Native multi-chain bridge	contracts/InayaTokenBridge{Home,Spoke}.sol, InayaChainRegistry.sol, InayaMessenger.sol; src/lib/chain-adapters/; deployments/bridge/*.json; aptos/programs/inaya-bridge-aptos/, sui/programs/inaya_bridge_sui/
Wormhole interoperability layer	src/lib/chain-adapters/interop/, src/app/api/interop/; deployments/interop/wormhole-wtt/bscTestnet-attestation.json; docs/{interoperability-provider-evaluation,inaya-interoperability,multichain-support-matrix,chain-expansion-guide,interop-security-boundary}.md
Storage backup & redundancy	contracts/InayaBackupRegistry.sol; src/lib/pinningProviders/ (Pinata + Filebase); docs/backup-redundancy-architecture.md
AI guarded execution + cryptographic audit trail	src/lib/ai-action-requests.js, src/lib/auditChain.js, src/lib/ai-business-tools.js (propose_* tools); src/app/api/cron/execute-approved-ai-actions/; src/app/api/orgs/audit/
Backend + web dApp + Business Workspace	inaya-network-dapp/src/app/, src/lib/, src/app/api/
Business Operations (Tasks/CRM/Procurement/Inventory)	src/lib/{task,deal,purchase-request,purchase-order}-workflow.js, inventory.js; src/app/api/orgs/{tasks,crm,procurement,inventory}/
Finance & HR	src/lib/{invoice,expense,employee,leave}-workflow.js, attachments.js; src/app/api/orgs/{finance,hr}/
RAG infrastructure	src/lib/rag/ (chunking, embeddings, vectorStore, retrieve, ingest); src/app/api/admin/rag/, src/app/api/cron/rag-reingest/
Mobile app	inaya-mobile/src/

AREA	PATH
Desktop — Business Workspace wrapper	inaya-desktop/src-tauri/
Desktop — dApp wrapper	inaya-dapp-desktop/src-tauri/

SECTION 23

Business Operations — Tasks, CRM, Procurement, Inventory

Four modules added to the Business Workspace on the exact same org/department/permission foundation Section 08 describes — no new auth system, no new storage system, every record department-scoped through the same canAccessDepartment gate every document already goes through.

MODULE	CORE OBJECTS	NOTABLE MECHANIC
Tasks	tasks (assignee, dueDate, status)	Real state-machine workflow, not just a status field — the {from,to,requiresManage,activityAction} transition-table pattern this whole layer reuses everywhere.
CRM	crm_contacts, crm_deals	A contact IS the unified Lead/Customer record — type flips LEAD→CUSTOMER on conversion rather than the record being recreated, so history/notes/deals stay attached.
Procurement	purchase_requests, purchase_orders, suppliers	PR→PO approval chain; PO receiving genuinely moves real Inventory stock — the two modules are integrated, not just adjacent.
Inventory	products, warehouses, stock_levels, stock_movements	stock_levels is a materialized view, only ever changed via $inc; stock_movements is the real append-only ledger — same 'ledger is truth' discipline used elsewhere in this codebase (faucet lifetime-caps, Finance's leave balances).

Shared infrastructure, not per-module reinvention:

- org_activity — one domain-generic audit log every module's transitions write into, not a bespoke activity table per module.
- getAccessibleScope() (document-permissions.js) — the one function that resolves 'everything this member can see across the whole org,' extended incrementally as each module shipped rather than four separate resolvers.
- AI tools: list_tasks, list_contacts/list_deals, list_suppliers/list_purchase_orders, list_products — all permission-scoped through the same getAccessibleScope() the human-facing routes use, so the Business Assistant never sees more than the requesting user could see themselves.

> Full detail (data model, workflow tables, permission model, test coverage, explicit out-of-scope list) in BUSINESS_OPERATIONS_TASKS.md, _CRM.md, _PROCUREMENT.md, and _INVENTORY.md at the repo root.

SECTION 24

Finance & HR Layer

Invoices, Expenses, Payments, and CSV reporting (Finance); Employee records, Employee documents, Leave management, and Department Administration (HR) — a testnet demonstration/validation layer, explicitly not regulated banking, tax filing, or payroll processing. Every Finance/HR screen carries a visible 'Testnet / Beta' badge.

Roles are additive, not a restructure.

org_members.role stays a single fixed string (owner|admin|member), checked in exactly one place (canManageOrg). Finance and HR each get their own new, OPTIONAL fields on the same document instead: financeRole (null|manager|staff), hrRole (null|manager|staff), managedDepartmentIds (new — Department Manager, a role that didn't exist before this layer). Zero risk to any existing module's gates.

CONCEPT	MECHANIC
Invoice lifecycle	DRAFT→SENT→PAID/CANCELLED, plus a cron-driven SENT→OVERDUE (invoices-mark-overdue, nightly, CRON_SECRET-gated) — the one transition in this entire layer that isn't user-invoked. Idempotent: a PAID invoice with a past due date is never touched, because the cron filters on status, not on date.
Expense approval	submit→PENDING_APPROVAL→APPROVED/REJECTED, approve/reject gated by canManageFinance (Finance Manager or org owner/admin) — Finance Staff can submit but not approve their own or anyone else's.
Employee identity	employees.memberEmail is nullable — set when the employee has real workspace login, absent when HR is just tracking someone who never logs in. Either way is a valid record.
The 'Employee' role	Not a role string at all — any member viewing the employees record whose memberEmail matches their own session gets read access to that one record and their own leave requests. A data-scoping rule, not an enum value.
Leave balance	Computed fresh on every read (allocationDays − this year's APPROVED day-spans), never a mutable stored counter — same 'ledger is truth' discipline as Inventory's stock levels (Section 23).
HR/expense documents	A new attachments collection, deliberately NOT a reuse of org_documents (which mandates a projectId and a department/project permission model wrong for 'who can see this employee's contract') — reuses the same client-side encrypt/shard/pin pipeline, just records metadata differently.

A real bug this layer's own tests caught. getAccessibleScope() originally filtered a Department Manager's managedDepartmentIds against their own already-visible departments before querying, intended as a redundant-query optimization — but department-level visibility doesn't imply HR/employee visibility, which is gated separately. A Department Manager whose own membership already included their managed department (the normal case) ended up with an empty managedDeptIds and silently lost their HR grant. Caught by test/hr-workflow.test.mjs's Department Manager scope test, fixed by removing the incorrect filter — a genuine finding from testing against real Atlas, not a hypothetical.

Full detail in BUSINESS_OPERATIONS_FINANCE.md and _HR.md at the repo root.

SECTION 25

RAG — Retrieval-Augmented Generation Infrastructure

The shared retrieval layer behind the Docs, Security, and Learn assistants (Section 12) — replaces the Docs Assistant's old static hardcoded knowledge block with real semantic + keyword search over a live, re-ingestable content index. Built on this project's real MongoDB Atlas cluster, not a bolted-on vector-DB vendor.

1. **Chunk & embed.** Static sources (docs pages, FAQs, the fundraising-docs content files, the security policy explainer, Learn's curated config, and lazily-ingested YouTube video transcripts) get chunked by heading/paragraph/structured-section as appropriate, then embedded via Gemini's gemini-embedding-001 (768 dimensions), content-hash-cached so re-ingesting unchanged content costs nothing.
2. **Index — native Atlas, both kinds.** rag_chunks carries both an Atlas Vector Search index ($vectorSearch) and an Atlas Search index ($search), both created idempotently in code (collection.createSearchIndex()) — no manual Atlas dashboard step required.
3. **Hybrid retrieve.** retrieveContext() merges vector + keyword results via Reciprocal Rank Fusion in application code (not a native $rankFusion stage, for broader Atlas-tier compatibility), then gates on a relevance threshold calibrated against real measurements on this project's own cluster — DEFAULT_MIN_RELEVANCE = 0.8, set after live-measuring relevant queries scoring ~0.89-0.91 cosine similarity against irrelevant ones at ~0.72-0.77 (a genuinely higher baseline than intuition suggested, and a real bug caught: an earlier 0.55 threshold plus a keyword-match override let an irrelevant query pass).
4. **Attribute the answer.** Every assistant response using RAG-retrieved context appends a formatted attribution of which source(s) it drew from — never a bare answer with no traceable origin.

Permission-safe by construction:

- Nothing private is ever ingested into the shared rag_chunks index — Security's per-identity events and Learn's per-wallet progress stay live, permission-scoped MongoDB tool calls (Section 12/23's pattern), never embedded text.
- custody-sdk's own docs and the legacy KNOWLEDGE_ARTICLES collection are deliberately excluded from ingestion — documented scope trims, not oversights.

Ops:

- Admin UI at /admin/rag (reingest, stats) plus a nightly cron (rag-reingest, CRON_SECRET-gated, same pattern as every other cron in this codebase).
- rag_query_log and rag_ingestion_runs give an honest record of what was actually retrieved and when content was last (re)indexed — metrics recording is fail-open, never blocks the caller.

> Full detail in RAG_INFRASTRUCTURE.md at the repo root.

SECTION 26

Multi-Chain Bridge & Interoperability (September 2026)

$INAYA is no longer BSC-only. Two independent systems move it to other chains, deliberately kept separate — a problem in one says nothing about the other. Every capability claim below is backed by a real transaction, not a deployment assumed to work.

System 1 — Inaya's own native bridge (mint/burn, BSC home)

InayaTokenBridgeHome (BSC) + InayaTokenBridgeSpoke per remote chain, coordinated by InayaChainRegistry (trusted-chain + spoke-address registration) and InayaMessenger, secured by threshold validator signatures over each cross-chain message. bridgeOut() on the source chain, executeMessage()/receive_message() on the destination.

CHAIN	VERIFIED LEVEL	EVIDENCE
BSC Testnet (home) / Sepolia / Avalanche Fuji	Staking-level — full unified staking interaction proven	Established prior to this pass; re-confirmed alongside the new chains below.
Arbitrum Sepolia	Messaging — registries wired and trust-verified, no transfer confirmed through it yet	deployments/bridge/arbitrumSepolia.json
Solana Devnet	Token-transfer — first-ever proven cross-chain message execution on Inaya's Solana program	Real bridgeOut + receive_message cycle, wrapped balance confirmed on-chain. deployments/bridge/solanaDevnet.json.
Hedera Testnet	Token-transfer	Hedera runs the EVM natively, so the identical spoke bytecode already proven on Sepolia/Fuji/Arbitrum was deployed unchanged. Real bridgeOut + executeMessage cycle confirmed. deployments/bridge/hederaTestnet.json.
Aptos Testnet	Token-transfer	New native Move contract (aptos/programs/inaya-bridge-aptos), reusing the same validator-signing scheme as the EVM spokes. Real cycle confirmed first attempt. deployments/bridge/aptosTestnet.json.
Sui Testnet	Token-transfer	New native Move contract (sui/programs/inaya_bridge_sui). Real cycle confirmed twice. deployments/bridge/suiTestnet.json.
Polygon Amoy	Discovered only	Configured, never deployed — not claimed as more.

> **Two real, non-obvious bugs found getting Solana and Sui working.** Solana: its native secp256k1 precompile keccak256-hashes the instruction's message field internally before recovering the signature — undocumented in the public client crate docs, root-caused empirically by tracing the real (and, confusingly, doc-versioning-inconsistent) error-enum ordering; fixed in the off-chain validator signing procedure, not the on-chain program. Sui: ethers.toBeArray(0) returns a zero-length array (minimal big-endian encoding of zero has no bytes), silently truncating a signature by one byte whenever a validator's recovery id was 0 — fixed in the relayer script.

System 2 — Wormhole interoperability layer (parallel, does not replace System 1)

Reaches other chains through Wormhole's third-party guardian network instead of a hand-built bridge per chain. Evaluated against LayerZero on live docs (docs/interoperability-provider-evaluation.md); Wormhole selected (WTT/lock-and-mint mode), LayerZero deferred rather than rejected.

4 real, proven routes (full lock-and-complete cycle, verified non-zero destination balance):

- BSC → Ethereum Sepolia, BSC → Arbitrum Sepolia, BSC → Avalanche Fuji — all proven end-to-end.

- BSC attestation-only path additionally proven.
- Solana: wrapped token created and verified, but a transfer is blocked because Wormhole's own guardians never registered BSC as a trusted source chain on their Solana Devnet Token Bridge — a governance action on Wormhole's side, not fixable by Inaya.
- Sui and Aptos via Wormhole: both blocked by stale bytecode/package references inside Wormhole's own SDK, confirmed by re-running the actual attempts and inspecting the exact on-chain abort — the same root cause that led directly to building Aptos/Sui support into the native bridge (System 1) instead.

> /bridge's UI offers only the routes actually proven as real send options; every other Wormhole-reachable chain is shown as reference-only, per an explicit no-fake-chain-support policy. Relay infrastructure (api/interop/wtt/{initiate,relay,status}, a 5-minute cron) fetches the Guardian-signed VAA via a plain REST call to Wormholescan's public API rather than the @wormhole-foundation/sdk package directly — importing that SDK inside a Next.js API route was found to break the production build (it eagerly pulls in XRPL support, whose dependency chain ships ESM-only files webpack can't bundle).

SECTION 27

Storage Backup & Redundancy (September 2026)

Closes a real gap that existed until this pass: each file's two encrypted shards (Section 05) were each pinned to exactly one provider (Pinata). Losing that one pin permanently lost the file — zero redundancy. Additive only; the existing encryption/sharding pipeline is unchanged.

1. **Replica redundancy, two independent providers.** Every shard is now replicated across Pinata and Filebase (an S3-compatible, IPFS-backed provider on a genuinely different infrastructure/failure domain), not just one.
2. **Two-tier failure detection.** Cheap pin-status checks every 15 minutes with a 3-strike grace window (a single blip never triggers recovery), plus daily content-hash integrity checks that catch real corruption with zero grace.
3. **Five backup-health states, a pure state machine.** Protected / Rebuilding / Degraded / Recovery Required / Recovery Failed — computed by a fully unit-tested state machine (19/19 tests), not scattered conditionals.
4. **Integrity-verified automatic recovery.** Fetches from a surviving healthy replica, verifies its hash against what was captured at original pin time, re-pins to restore the target replica count, and deletes the failed replica's underlying storage object (not just its database record).

COMPONENT	DETAIL
InayaBackupRegistry.sol	Deployed to BSC Testnet: 0x062c341aE4f11CB1dEa1B0D3930d52902F97f48a. Records only a redundancy commitment and health-state transitions on-chain — never the replica data itself — written only at real state boundaries, mirroring InayaProofRegistry's trust model. 15/15 contract tests.
InayaKernel.Backup SDK client	getBackupStatus / getBackupHealth / getRedundancyStatus / getRecoveryStatus / requestRecovery. 9/9 tests, published to npm as part of custody-sdk.

Real, live, end-to-end proof — not just unit tests. Replicated a real already-uploaded asset's shards to both Pinata and Filebase (correctly computed Protected, matching an independently-confirmed on-chain transaction); then deleted a real Filebase replica and watched the system correctly detect it (Recovery Required), fetch from the surviving Pinata copy, verify its hash, re-pin to Filebase, and self-heal back to Protected — with the on-chain registry tracking every real state transition. Two real coordinator-level bugs (not in the already-correct, already-tested state machine itself) were found and fixed by this exact proof run: a recovery sweep that checked shard health using a fake empty replica list, always reading worst-case; and a source/target-provider selection rule requiring zero recorded failures ever, instead of the same 'still retrievable' definition used everywhere else in the system.

One real, external, currently-open constraint: the project's Pinata account is over its plan's usage limit (confirmed via a live 403 from Pinata's own API), which blocked the first attempt at a fresh live pin during this proof and separately affects custody-sdk's own release-pinning step (Section 05). Not a code bug — an account-side limit to address separately. Full writeup: docs/backup-redundancy-architecture.md.

SECTION 28

AI Guarded Execution & Cryptographic Audit Trail (September 2026)

The Business Assistant's most consequential capability: it can propose real changes across 9 business domains (Task, Document, Deal, Purchase Request, Purchase Order, Leave Request, Employee, Expense, Invoice) — and can never execute any of them itself. src/lib/ai-action-requests.js is the guarded-execution state machine; src/lib/auditChain.js is the tamper-evident log everything writes into.

State machine (ai-action-requests.js)

PENDING_APPROVAL → APPROVED (sets unlockAt = now + 36h) → QUEUED (cron, once unlocked) → EXECUTED (cron, after calling the real transitionX()) or EXPIRED (cron, if the real transition no longer applies). PENDING_APPROVAL can also go to REJECTED; APPROVED can go to CANCELLED, but only before unlockAt passes.

Authorization, belt-and-suspenders throughout:

- proposeAiAction() requires canPropose — resolved by the caller through the exact same getAccessibleScope()-derived gate the real action's own tool context uses. An AI can't propose something the requesting user isn't already allowed to ask for.
- reviewAiAction()'s approve/reject requires canApprove — resolved through the domain's real requiresManage gate (e.g. canManageFinance for an expense decision). An AI-proposed action can never be approved by someone who couldn't already perform the real action themselves.
- Execution (the 36h-later cron) runs as a synthetic org-manager membership attributed in the activity log to the human who approved it — not a fabricated identity, and not re-derived from the approver's current membership (which could have changed since approval).
- Idempotency: idempotencyKey = sha256(orgId+toolName+targetRecordId+args+hour-bucket), uniquely indexed — an identical proposal within the same hour upserts into the existing pending request instead of duplicating.
- Risk classification (classifyRisk()) is keyed first by exact action, falling back to a per-domain default, and never returns undefined — an unrecognized combination defaults to MEDIUM rather than silently under-classifying as LOW. EXPENSE, INVOICE, and PURCHASE_ORDER default HIGH; PURCHASE_REQUEST approve/reject and EMPLOYEE terminate are explicitly HIGH.

The audit trail (auditChain.js) — a hash-chained overlay, not a replacement log

Runs alongside the existing plain-insert logs (org-activity-log.js, activity-log.js, security.js), which keep writing exactly what they always have. This module additionally appends a linked entry: entryHash = sha256(prevHash + canonicalJSON(eventFields)) — each entry commits to the entire chain before it, not just its own content, so altering or deleting any past entry breaks every hash after it.

- Concurrency: one org's chain is strictly sequential (entry N must know entry N-1's real, committed hash), so appendAuditEntry does an optimistic compare-and-swap on audit_chain_heads and retries on conflict — under contention this means a few wasted reads, never a corrupted chain; a conflicting write always fails closed.
- verifyChainIntegrity(orgId) walks the whole chain and recomputes every hash, catching a direct DB edit, a deleted entry, or a reordered entry — returns exactly which sequence number broke and why.
- GET /api/orgs/audit/export (org-scoped, owner/admin only) exports the full chain — every field needed to independently recompute and verify it — so a business customer doesn't have to trust Inaya's own "Verified" banner; they can walk the export and recompute sha256(prevHash + canonicalFields) themselves. Same shape as the internal admin export, just gated by org membership instead of internal auth.

> **Why this belongs next to custody-sdk's release verification (Section 05).** Same underlying principle applied to a different trust boundary: custody-sdk lets anyone verify the client code wasn't tampered with before it ran; the audit trail lets anyone verify a business's own activity log wasn't tampered with after it was written. Neither asks the reader to simply trust Inaya's word. 19 automated tests cover this system, 11 of them adversarial scenarios specifically trying to defeat the guardrails (approving without real authority, bypassing the delay, replaying an idempotency key, etc.) rather than just the happy path.

SECTION 29

Web3 App Store, NFT Vault & Wallet-Attack Protection (September 2026)

Three related additions to the dApp side, shipped 2026-09-01: a curated + community app directory, an NFT backup vault, and live threat-registry checks wired into the bridge's recipient field.

Web3 App Store (/apps) — real submission-to-approval pipeline, not just a directory

8 first-party apps plus dynamically-fetched community listings (src/lib/appStoreListings.js). A developer submits either an already-pinned IPFS CID (opens via a public gateway in a new tab, never touches Inaya's origin) or an externally-hosted URL rendered in a strictly sandboxed iframe (src/app/apps/embed/[slug]/page.js — no allow-same-origin combined with allow-scripts, the actual dangerous combination). Same-origin hosting and an unvetted registry were both explicitly considered and rejected.

- Every submission requires a wallet signature (the same generic metadata-auth framework used elsewhere in the codebase) and is checked against the live Security Layer threat registry both at submission and again at admin-review time — nothing is ever auto-published, every listing starts pending.
- Admin review/approve/reject queue at /admin/app-store, mirroring /admin/audit's existing passphrase-gated session pattern exactly.
- Verified end-to-end with a real signed submission, a real reject/approve cycle, and three adversarial cases (malformed CID, javascript: URL scheme, forged wallet signature) — all correctly rejected. Two real bugs were caught and fixed during that verification: /apps was being statically prerendered at build time (new listings would never appear until redeploy), and a resourceId mismatch between the raw signed URL and its normalized form was incorrectly flagging legitimate submissions as tampered.
- Genuinely missing, not yet built: a CLI deploy path directly into this store (custody-sdk's CLI deploys to the storage network, not to this app-store collection).

NFT Vault (/nfts)

Discovers a wallet's owned tokens from a specific ERC-721 + Enumerable collection (no indexer credentials configured, so auto-discovery across every collection is a stated, honest gap, not silently claimed) and backs up metadata + image via the existing encrypt/shard/pin pipeline (src/lib/clientCrypto.js). A backup record requires both a wallet signature and a real on-chain ownerOf() check — API at src/app/api/nft/backup(s)/route.js.

Wallet-attack protection — live threat check on the bridge's recipient field

A real browser extension doesn't exist and wasn't judged proportionate; instead the existing Security Layer threat registry (Section 09) is wired directly into the Bridge page's recipient address field (src/components/AddressRiskCheck.js) — a debounced live check, honest silence when there's no data on file, and a clear warning for a real CONFIRMED/DISPUTED report. Verified live end-to-end against a seeded threat record.

SECTION 30

Multi-Factor Authentication & September 2026 Security Hardening

Two related trust-surface additions: real MFA on Business Workspace login, and a full ecosystem security audit + remediation pass across web, mobile, custody-sdk, and both desktop apps.

MFA (TOTP + SMS) on Business Workspace login

An optional second factor on top of the existing magic-link/Google sign-in — TOTP (any standard authenticator app) or SMS via Firebase Phone Auth, managed from the Security nav item (src/components/business/MfaSettings.js) and enforced at login (MfaVerifyScreen.js). Mirrors the same pattern on inaya-mobile.

Security hardening pass — 2 critical, 6 high, 8 medium findings, all remediated

A full audit across the whole ecosystem (web, mobile, custody-sdk, both desktop apps, node infra). Selected findings, chosen because they're the ones a reader would most want confirmed fixed rather than just counted:

- Critical: a hardcoded Aptos private key in a relayer dry-run script — removed, the script now reads it from env like every sibling relayer script, and the key itself has been rotated (not just removed from source).
- Critical: Next.js bumped 14.2.5 → 14.2.35 (patched) plus a protobufjs override, closing a critical transitive advisory — 0 critical vulnerabilities remaining as of this pass.
- High: /api/upload was an unauthenticated, unmetered proxy to Inaya's own storage backend — now requires a real session and is rate-limited.
- The remaining high/medium findings span rate limiting, security headers, desktop IPC hardening, and general key-hygiene fixes across inaya-desktop/inaya-dapp-desktop and scripts/.

> Same honesty convention as the rest of this document: every finding above cites a real fix in a real commit, not a general claim of "hardened." A public developer-facing overview of the custody-sdk ecosystem, pulled from the SDK's own README/SDK_GUIDE rather than separately maintained, is also now live at inayanetwork.com/build ("Build on Inaya") for anyone wanting the install commands and quickstart directly.

SECTION 31

Sovereign Enterprise OS — The Connecting Layer (September 2026)

Most of what this SOW asked for already existed — Business Workspace, AI Guarded Execution, the audit trail, the App Store, custody-sdk. What was genuinely missing was the layer connecting all of it into one coherent operating environment rather than a pile of separate features. Built on both surfaces as two parallel instances sharing the same underlying engines: Business Workspace (email/org identity) and the main dApp (wallet identity) are confirmed, completely separate identity systems today — no walletAddress field exists anywhere on an org_members record — so this deliberately does not build a new identity-linking layer, it builds the same connecting capability twice from shared code.

Identity Context — the one foundational gap everything else depended on

Neither surface had a client-side identity context before this pass; both prop-drilled session/wallet state through one top-level component (BusinessPage→Workspace, and page.js's single ~7500-line Home()). New src/contexts/OrgContext.js and WalletContext.js (React Context, mirroring ThemeProvider.js's shape — the only prior Context in the codebase) make identity reachable via useOrg()/useWallet() from any component, zero props threaded through.

> orgs.js's permission gates (canManageOrg, canAccessDepartment, canManageFinance, canAccessFinance, canManageHR, canAccessHR, isDepartmentManager) were extracted into a new client-safe module, src/lib/orgGates.js, so OrgContext could import them at all — orgs.js itself imports node:crypto and the mongodb driver, which can't ship to a browser bundle. orgs.js re-exports every one of them under its own name, so all ~60 existing server-side `import {...} from "./orgs.js"` call sites needed zero changes.

Cross-Module Trust & Health

Confirmed nothing anywhere combined audit-chain integrity, AI action backlog, backup health, and security status into one signal (grepped for trustScore/systemHealth/aggregat* — zero hits). New src/lib/trustHealth.js's computeTrustHealthSnapshot() does this for real, per scope — org-side combines verifyChainIntegrity(), pending AI actions by risk tier, and overdue business items (platform-wide security stats are explicitly labeled as such, not presented as org-specific, since security.js has no org linkage); wallet-side combines the wallet's own recent security events with real per-asset backup health across its own files, a cleaner signal than the org side since security/backup data is already wallet-keyed.

PIECE	WHAT'S GENUINELY NEW	REUSED, NOT REINVENTED
Unified Notifications	New notifications/notification_reads collections with real persistent read-state — the only prior 'notifications' were a narrow referral/KYC read-model with no DB collection at all, unread state only in localStorage.	Write-hooks from proposeAiAction/reviewAiAction, document-workflow.js's submit transition, and backupEngine.js's state-change boundary crossings — additive calls into already-tested functions, not rewrites.
Unified Search	New Cmd/Ctrl+K src/components/CommandPalette.js, src/lib/orgSearch.js + walletSearch.js — the only prior search anywhere was a scoped YouTube proxy for Learn.	getAccessibleScope() as orgSearch's only data source (no-leak-by-construction); metadata_files as walletSearch's source, the same off-chain enumeration list-files/route.js already documents as the only full-list source that exists.
Activity Center	New src/lib/activityCenter.js's generateWhatChanged() — genuine period-over-period counts (AI outcomes, notification volume) from real timestamped fields.	business-brief.js's generateBusinessBrief() output composed verbatim as the org-side business section, not reimplemented.

PIECE	WHAT'S GENUINELY NEW	REUSED, NOT REINVENTED
OS-Level AI Router	New src/lib/ai-os-router.js — a thin dispatch shim merging Business+Security (org) or a new search_docs tool (wraps RAG's existing retrieveContext, domain:"docs") + Security (wallet) behind one conversation, tool-name-prefixed so Gemini sees one collision-free list.	runBusinessTool/runSecurityTool called unmodified after prefix-stripping — a router bug can't change what either tool set actually does.
OS Home	New OsHomeView.js (Business Workspace, new default view) and OsHomeSection.js (dApp, new non-default tab — that page is the public anonymous landing page too, so its default couldn't change).	No new aggregate backend route — parallel fetches against the APIs above, same pattern DashboardView already uses for /api/orgs/dashboard.

Desktop multi-window

Both desktop apps could only ever open a single window before this pass — confirmed: tauri.conf.json has "windows": [] (built programmatically) and lib.rs constructed exactly one "main" WebviewWindowBuilder. New open_module_window(label, path) Tauri command (identical implementation in both apps' src-tauri/src/lib.rs) reuses the exact same WebviewWindowBuilder::new(app, label, WebviewUrl::External(...)) call the main window is already built with, plus each file's own verify_trusted_origin guard — not a new window-creation mechanism, just parameterizing the one that already existed.

> **Verified for real, not just type-checked.** A real compile bug was caught fixing this: naming url::ParseError explicitly in a map_err closure doesn't compile, since the url crate is only a transitive tauri dependency, not a direct one in Cargo.toml — fixed by letting type inference flow from context instead, the same thing the existing APP_URL.parse().unwrap() calls already lean on. cargo check passed cleanly on both crates; a genuine cargo build --release then succeeded for both via the existing scripts/build-portable-desktop.ps1 (unmodified — it already supports -App both/desktop/dapp), producing real standalone portable .exe files in roughly 1m35s each; both apps' existing build-desktop-linux.yml/build-dapp-desktop-linux.yml CI workflows ran green on the actual push, path-triggered automatically, not manually invoked.

> **What OS Home does and does not guarantee.** Guarantees: every number shown is a real value from an already-shipped, already-tested API — no new aggregate route fabricating a summary. Does not guarantee: a unified identity across the dApp and Business Workspace (a deliberate, confirmed-with-the-team scope decision, not an oversight — see the opening note above); a historical trust/health trend (the aggregator computes live, nothing persists snapshots yet, so Activity Center reports current-state rather than a claimed delta it can't honestly back).

SECTION 32

Healthcare & Legal OS Expansion (September 2026)

Extends the Sovereign Enterprise OS (§31) into two vertical specializations — Health OS and Legal OS — for organizations handling patient or client/matter data, on Business Workspace's existing org/email identity side only. Reuses the shared OS foundation throughout (getAccessibleScope(), the audit chain, notifications, unified search) rather than building parallel systems — confirmed via codebase audit before writing anything: appendAuditEntry() is fully generic and reused directly for healthcare/legal audit and evidence chain-of-custody by passing new recordType values, no second audit system built.

Assignment-based access — a genuinely different model from every other module

Department membership grants nothing here. Only an explicit care-team or matter-team assignment (new health_care_team_assignments/legal_matter_team_assignments join collections, mirroring project_members' shape) — or an org-wide healthRole/legalRole — grants patient/matter visibility, wired into getAccessibleScope() itself rather than a parallel resolver. Verified by tests: a department member with no assignment sees zero patients or matters, even though they can see everything else in that same department.

> **A real bug this caught.** Break-glass emergency access is implemented as a genuine, time-limited care-team assignment (not a separate bypass code path) — but the first version had no expiry check in getAccessibleScope()'s query, so a '4-hour' emergency grant would have stayed active forever. Caught by writing the expiry test, fixed with a notExpired filter, reverified. Every grant is still audited the instant it happens and flagged for manager review afterward.

Legal holds that actually block deletion, not just display a warning

retention.js's checkDispositionAllowed() is the guard every deletion call site checks first — verified to return allowed:false for any record tied to a matter under an ACTIVE hold (matched by direct record, matter, or custodian scope), and allowed:true again once released. Written ahead of legal-hold-workflow.js in Phase 1 specifically so every future disposition path gets this protection automatically the moment it's wired in, no second pass needed.

Trust accounting — one hard safety rule, no compliance claim

trust-accounting.js recomputes a matter's real trust balance and rejects any withdrawal that would exceed it, atomically, before the write — verified with a test attempting to overdraw a real seeded balance. Explicitly no jurisdictional (IOLTA-style) compliance validation is claimed or attempted; this is a financial-safety guard, not a regulatory one, per the SOW's own instruction not to claim compliance without jurisdiction-specific validation.

AI kept deliberately narrow — the capability isn't exposed, not just discouraged

ai-health-tools.js and ai-legal-tools.js are 100% read/summarize/draft — no propose_* mutation tool exists for either vertical, since SOW §10.28/§11.20's own tool lists are entirely read/summarize/draft to begin with. A free-text query framed as a diagnosis request, a request for final legal advice, or a request to release a hold or delete evidence is refused at the tool layer (a regex guard checked before any summarization runs), not left to a prompt instruction the model could be talked out of — verified with real refusal tests across both tool sets.

The vertical lock — real enforcement added after an explicit follow-up request

An org's configured business type (general/healthcare/legal) initially only gated which nav items business/page.js showed — a law firm could still call a Health OS API route directly and it would work. requireVertical() (industry-config.js) closes this: every one of 25 Health/Legal API routes now rejects a mismatched org with a real 403, checked immediately after authentication in every handler, fail-closed as 'general' for any org with no vertical configured at all.

A second real bug this caught. The automated multi-file edit that inserted requireVertical() into all 25 routes referenced a variable that didn't exist in scope in 14 POST handlers (they only ever destructured body.orgId, not a bare orgId) — a guaranteed 500 crash on every POST call to those routes. Caught by a full re-scan before committing, fixed, and backed by a new permanent test (vertical-lock-wiring.test.mjs) that statically confirms requireMembership() and requireVertical() are always called with the exact same expression across every one of the 25 route files — this exact bug class can't silently ship again. Beyond the test suite: live-verified against the running dev server with three real seeded orgs and real session cookies — a healthcare org's GET and POST both genuinely succeeded, a general org's identical calls were cleanly rejected with 403 (never a crash).

What's real backend but has no screen yet. Of roughly 20 domain modules across both verticals, only Patients (Health OS) and Matters (Legal OS) have an actual UI screen today — verified live in a real browser against real seeded data. Consent, ROI, break-glass, billing, scheduling, and research (Health OS) plus clients, prospects, evidence, holds, discovery, redaction, deadlines, contracts, entities, trust accounting, and time tracking (Legal OS) all have real, tested backend logic and a real, vertical-locked API route, but reaching them requires calling the API directly today. Nothing on inaya-mobile for either vertical yet. FHIR/HL7/DICOM/lab/pharmacy/claims/e-filing/legal-research/e-signature/SSO are documented adapter interfaces (adapterStub.js) with an honest configured:false stub, not live integrations — none of these can be genuine without the user's own third-party credentials. No HIPAA/ABA/eDiscovery compliance certification exists or is claimed anywhere.

SECTION 33

Financial Services & Regulated Enterprise OS — All Ten Phases (2026)

Extends the Sovereign Enterprise OS (§31) into a third and fourth vertical specialization — Financial Services OS (hedge funds/asset managers), Private Capital OS (PE/VC), and a shared Regulated Enterprise Control Plane (banks, insurers, pharma, and other cross-industry regulated organizations) — built in the same additive, reuse-first discipline as §32: appendAuditEntry(), getAccessibleScope(), and the guarded-execution shape from ai-action-requests.js are all reused directly rather than forked.

The Regulated Enterprise Control Plane came first, deliberately

Phase 0+4 (the compliance foundation — control library, evidence vault, findings, versioned policies, compliance exceptions, internal audit, and a Regulatory Examination Workspace with scoped one-time-use examiner links) was built before Financial Services OS or Private Capital OS's own domain modules, specifically so those two verticals build on one shared compliance layer rather than each reinventing it. compliance-policies.js's publish-immutable lifecycle (no updatePolicyDraft() path reachable once PUBLISHED — amendPolicy() always inserts a new version) is the single most load-bearing property of this phase, backed by a dedicated permanent test.

Entity-scoped, not org-wide, visibility

The Financial Entity Core (funds, entities, investors, counterparties) and Private Capital's deal pipeline both use assignment-based visibility (financial_fund_team_assignments), the same isFundTeamMember/isCareTeamMember/isMatterTeamMember pattern §32 established — a user does not automatically see every fund or deal in their org just by being a member of it.

Nine role-specific AI copilots, zero mutation tools

CIO/COO/CCO/CRO/CFO/GC/analyst/deal-team/security/auditor copilots all follow ai-compliance-tools.js's own precedent: 100% read/summarize/flag, with a request phrased to elicit a compliance certification or an 'are we compliant' verdict refused at the tool layer before any summarization runs, not left to a prompt instruction alone.

> **What's real backend but has no mobile screen yet.** All ten phases (Regulated Control Plane, Financial Entity Core, Investment Management, Private Capital, Security & Resilience, AI copilots, Integration Adapter Architecture — stub-by-default, configured:false until real credentials exist — Executive/Board Layer, External Data Rooms, and Enterprise Hardening) are live-verified end-to-end on the web app. Nothing on inaya-mobile for any of these three verticals yet — that gap is explicitly called out in the roadmap rather than left ambiguous. No compliance certification of any kind exists or is claimed anywhere.

SECTION 34

Government & Public Sector Sovereign OS — Phase 1 Foundation (2026)

A fifth vertical specialization, for government departments and agencies handling citizen, legal, financial, procurement, health, and operational data. Built the same way every prior vertical was: confirm what's already reusable first, build only the genuinely new part second. Most of §D's required scope (Procurement, Contracts, Finance, HR, Tasks, Approvals) needed zero new engineering — those modules were already vertical-agnostic.

Citizen Records reuse Health OS's exact access model

citizen-records.js's isCitizenRecordAssignee() is the same shape as isCareTeamMember() from §32 — a citizen record is visible only to someone actually assigned to it, never to anyone merely holding governmentRole:'staff' or department membership. Verified by a test proving a staff member with no assignment is denied even though they hold general Government OS access.

Need-to-know enforced twice — API layer and AI tool layer

ai-government-tools.js's search_citizen_records doesn't just trust that the API layer already checked access — it re-runs the same requireCitizenRecordAccess() check per record before returning anything, so an AI assistant can never surface more than the human it's acting for could see. Verified live: a search for a name that matches both an assigned and an unassigned record returns only the assigned one.

A stricter chain-of-custody rule, gated to one vertical only

Government-vertical orgs get every document READ logged to the audit chain, not just every write — a genuinely stricter bar than the general Documents module. Implemented as an additive, vertical-gated branch inside the existing retrieve route rather than changing that route's behavior for every other org.

> **What's real backend but has no mobile screen yet.** Citizen Records, Case Management, the Policy Knowledge Base (reusing compliance-policies.js's publish-immutable lifecycle), the operations/security dashboard, and the AI assistant are all live-verified on the web app — 51 new automated tests, including the two load-bearing properties (assignment-required access; publish-immutability) each written before the implementation they verify. Break-glass access needed zero new code: privileged-access.js was already built vertical-agnostic during §33. Nothing on inaya-mobile yet. No government certification, accreditation, FedRAMP authorization, or jurisdiction-specific compliance claim exists or is claimed anywhere — that requires separate authorities entirely outside this codebase.

SECTION 35

Institutional Trust Infrastructure (September 2026)

Five pieces that turn Inaya's existing audit-chain and permission machinery into something an outside party — an investor, an auditor, another organization, a developer — can actually reach and independently check, rather than something only visible from inside a logged-in session.

A public Trust Center with real client-side cryptographic verification

/trust is unauthenticated, like /security. Its centerpiece isn't a status badge — the verifier reimplements auditChain.js's exact hash formula (sha256(prevHash + canonicalJSON(sortedFields))) in the visitor's own browser via the Web Crypto API, then recomputes every entry's hash independently and reports the exact sequence number where a chain breaks, if one does. A tampered entry fails because SHA-256 is collision-resistant, not because the server said so. The page distinguishes two real guarantee strengths and states which one the visitor got: a full export from chain genesis proves nothing was ever altered from the very first recorded event; a scoped, single-record export (from the new public evidence API below) proves only that those entries weren't altered after export, not what came before them. No SOC 2, HIPAA, ABA, or FedRAMP certification is claimed anywhere on the page — it states this directly.

A real bug the verifier itself caught

The first version of the client-side verifier assumed every export started at chain genesis. A legitimately scoped, single-record export doesn't — its first entry's prevHash correctly points at a real prior entry the scoped export doesn't include, which made the original verifier falsely report a perfectly valid export as broken. Found live while verifying a real resilience-test evidence package (§36), fixed by having the verifier auto-detect which case it's looking at and apply the matching guarantee — the kind of failure mode only surfaces when two systems that were each individually correct meet for the first time.

Cross-organization trust — a primitive, not yet a live data-sharing path

org-trust.js gives two organizations a real propose → accept/reject → revoke workflow with a scoped permission list, an expiry, and a query function (isTrustedAccess) any future feature could call to check whether org A currently trusts org B for a given scope. Today, nothing in the codebase actually calls that query function yet — this ships as bookkeeping infrastructure for cross-org data sharing, not a live data-sharing feature itself. Independent control is structural: the accept/reject database filter only ever matches rows where the caller is the recipient, so an org can never approve its own outbound proposal no matter what ID it supplies.

An API-key developer platform, scoped honestly

api-keys.js issues a raw key exactly once at creation — never stored or recoverable afterward, only its hash is kept — and resolves every request to a synthetic owner-level membership bound permanently to that key's own org; no request parameter can ever redirect a key to act as a different organization. Three public/v1 routes sit behind key auth: audit/verify (chain integrity for the key's own org), evidence (a single record's trail — both recordType and recordId are required, so this is a per-record lookup, never an org-wide dump), and permissions/check, which checks whether a named capability is enabled for the org, not an individual user's personal role — a key always carries full organizational authority, and the endpoint's own documentation says so rather than implying per-user RBAC it doesn't do.

One evidence trail, one cross-vertical attention query

evidence.js doesn't introduce a new data store — it's a read-only merge of the existing human activity log and any linked AI action requests for one record, into one chronological timeline. institutional-attention.js does the equivalent for a person: it re-checks pending approvals against the real approval gate, pulls the same overdue-tasks and pending-documents tools the AI assistants already use, and returns one sorted list — but only as an AI-assistant tool today, not a REST endpoint a script could poll directly.

Every route here is testnet-only — grep across all eleven files and routes in this section returns zero mainnet references, consistent with the rest of the platform. Per-org resilience status and recovery-test results are deliberately never published on the public Trust Center, to avoid disclosing one customer's specific security posture — they stay inside the authenticated Business Workspace. Every module ships with a co-located automated test file, none of it mocked against a fake database.

SECTION 36

Autonomous Resilience Layer (September 2026)

Recovery capability that's continuously, automatically re-tested against real infrastructure — not backup existence treated as a proxy for recoverability, and not a claim that's only checked when someone remembers to check it.

The core constraint: real customer vaults can never be part of an unattended test

Reconstructing a real vault needs the real user's passkey, which Inaya never stores server-side — so no scheduled job can ever touch real customer data. The resolution: the resilience layer owns small synthetic 'canary' documents, pushed through the exact same production pipeline real uploads use (the same client-crypto sharding, the same pinning-provider adapters, the same replication and reconstruction functions) — never a mock or a stand-in copy of that pipeline. The canary's own synthetic passkey is the one deliberate, explicitly-documented exception to Inaya's 'never store a passkey server-side' rule, safe only because a canary is never real customer data.

A 13-step test against real dependencies

Each run selects a policy's critical assets, records a start time, invokes the real reconstruction primitive, validates content-hash integrity, checks real replica health and real permission-boundary behavior, times the whole thing with a wall clock, and writes the result to the same audit chain every other evidence-generating action in the app uses. Actual RTO is real wall-clock duration; actual RPO is honestly defined as minutes since the canary's most recently confirmed-healthy replica check — the real staleness of redundancy confirmation, never a fabricated number.

Scheduled, dashboarded, and AI-queryable — never AI-mutable

A daily cron sweeps every org whose test window has elapsed; a manual 'Run now' exists for on-demand checks, distinctly tagged so the evidence trail always shows which triggered which. A pure, unit-tested function derives one of five states (Unknown / Test Due / Verified / Failed / Degraded) from real test-run data alone — a paused policy always reports Unknown, regardless of history. The AI assistant gets four read-only tools (status, last result, failed assets, plain-language failure explanation grounded only in the real test record) and exactly one tool with a real side effect — trigger a test now — deliberately kept outside the Guarded Execution approval pipeline because it can only ever start the same non-destructive synthetic test the cron already runs unattended. The assistant's own system instruction explicitly forbids declaring compliance, claiming an unverified recovery capability, or fabricating a result.

> Verified live against real Filebase network calls during development. Recovery-test evidence uses the exact same verifiable evidence-package model as the Trust Center (§35) — no separate, weaker proof format was built for this. Zero mainnet references anywhere in this module.

SECTION 37

Node Operator Dashboard (September 2026)

A dedicated, wallet-authenticated home for node operators at /operator, entirely separate from the dApp's own wallet context and the Business Workspace's email-based session — seven tabs: Overview, Uptime & Telemetry, Qualification, Tier & Rewards, Events, Network, and an optional multi-node Fleet view.

A new, deliberately shorter-lived session layer

Every prior node route authenticated one wallet-signed action per request; nothing persisted a browser session. The dashboard adds a real session on top of that same unchanged signature-verification logic — a 12-hour cookie, deliberately shorter than the Business Workspace's 30-day session because it gates commission and reward data. Login requires the wallet already be a registered node; every route resolves data strictly from the session's own wallet, never from a client-supplied node ID.

Uptime history that refuses to backfill what it doesn't have

An hourly cron populates a new snapshot collection going forward from the feature's ship date. If a requested window (say, 90 days) reaches further back than the earliest real snapshot, the response honestly reports the shorter real coverage window and flags insufficient data, rather than estimating or padding a fuller-looking history. The qualification tracker (90-day, 95%-uptime streak) applies the same discipline day by day — a day with zero recorded snapshots moves the streak nowhere, in either direction.

Real on-chain reads, with an honest accuracy trade-off disclosed

Tier and commission are read live from the same deployed node-registry contract the existing settlement-release cron already relies on — explicitly the source of truth, since the off-chain tier field on a node's own record is set once at registration and never revisited. Every chain read fails honestly (an explicit 'could not reach the chain right now' response) rather than fabricating a tier. The one disclosed simplification: the public, no-login Network tab shows tier distribution from off-chain records, not a live per-node chain read for every operator — documented in the code as a real accuracy trade-off made for scalability, not an oversight.

> No claim/release button exists anywhere in the dashboard — settlement release stays fully automated by the existing timelock cron; the dashboard can only ever show what that timelock allows. This was treated internally as the highest-risk new surface added in this pass and is tested with real wallet signatures, not stubs.

SECTION 38

Four New Business Primitives: Sign, DePIN Storage, Escrow, Attestation (September 2026)

Four new Business Workspace modules, each with a module-header comment stating plainly what's real and what isn't — verified against the actual code, not just the comments. The throughline across all four: real cryptography and real ledger effects where they exist, and an explicit refusal to claim a stronger guarantee (on-chain, zero-knowledge, non-custodial) than the code actually provides.

Inaya Sign — real signatures, an optional real on-chain anchor

A signature binds signer, document version, document hash, and timestamp; re-signing a document that changed after the request was created is rejected because the current file hash no longer matches the hash the request was bound to. Wallet signers get a genuine ethers ECDSA signature check over a canonical message, the same convention the rest of the app already uses for wallet auth; signers without a wallet get an explicitly labeled session-consent record, never presented as cryptographically equivalent. Once a request is fully signed, an optional step computes a hash of the completion certificate and anchors it in a real transaction on the same already-deployed custody contract the Documents module registers hashes on. The individual per-signer proof stays off-chain and independently verifiable by anyone; only the completion certificate is optionally anchored — described precisely to avoid implying on-chain signatures that don't exist.

Native DePIN Storage Manager — a real control plane, not yet connected to the node fleet

Enterprise storage-node registration, health reporting, and eligibility tracking are real and additive — confirmed the public node-operator program has no concept of an organization at all, so this is genuinely new infrastructure, not a repaint of it. But every registered node's eligibility is a static, honest 'routing not yet supported,' because the storage backend has no routing-by-node concept yet; nothing here moves an actual shard. What is real: allocation and replica-health figures are pulled from the same existing plan and backup-engine subsystems every other storage view already uses, not recomputed or invented for this screen. Declared regional preferences are additive fields on the existing data-residency policy, always labeled Declared, never Active, since no pinning provider routes by region yet.

Milestone Escrow — an internal, approval-gated release workflow, not an on-chain escrow contract

The module's own header states this directly: there is no two-party on-chain escrow contract in this codebase, and the feature is never described as non-custodial on-chain escrow anywhere it appears. A milestone moving to 'confirmed' does not release funds — it only creates a Guarded Execution proposal, the same approval-plus-36-hour-delay pipeline used everywhere else in the app, requiring a separately authorized approver. Only after that delay does a cron execute the one function that actually moves money, writing a real, ledger-linked payment record — re-validating the milestone isn't disputed at execution time, since state can drift during the 36-hour window. A dispute hard-blocks release even after approval and requires separate authorized resolution.

Cryptographic Financial Attestation — a real hash commitment, explicitly not a zero-knowledge proof

The module's header names the exact ZK libraries it confirmed are absent from the codebase's full dependency tree, root and custody-sdk, to avoid the label being applied loosely. What it actually does: pull an org's real, permission-scoped, currency-converted financial records for a period, evaluate one narrow statement (revenue at least X, or expense at most Y) server-side, and hash-commit the canonical dataset. Verification independently re-runs the entire query against current data and recomputes the commitment — if a single record changed since generation, the hash drifts and the result is invalid, never silently passed. Only two statement types are implemented; a third (solvency) is explicitly rejected rather than approximated, because the codebase has no assets/liabilities data model to honestly attest against. A real nested-object canonicalization bug was found and fixed while building this feature's tests, and the same fix was applied to Inaya Sign's anchor certificate, which shared the identical flaw.

All four are wired into the Business Workspace as real nav entries backed by real, org-scoped API routes — not placeholder screens. Roughly 19 new routes across the four modules, each with its own test suite (inaya-sign, milestone-escrow, depin-storage-manager, financial-attestation).

SECTION 39

Business Workspace Operational Enhancements & Guided AI Assistant (September 2026)

Finance & Procurement — real records, honestly described mechanisms

Invoice 'PDF' generation is a print-styled HTML document with a Print/Save-as-PDF control, not a server-generated binary — the route's own comment confirms no PDF-generation library exists anywhere in the app, and the browser's native print-to-PDF already covers both use cases. Multi-currency (USD/EUR/GBP/AED/PKR) is real and server-validated on every invoice and PO, but conversion runs off a static, explicitly dated reference-rate table, not a live FX feed — the original amount and currency are always what's stored; conversion happens only at display time. Warehouse transfers are fully real: two ledger movements plus a materialized balance update sharing one transfer ID, with a real compensating reversal if the second leg fails after the first committed. PO editing and its filters are real CRUD, restricted to draft-status orders by an atomic database guard, not just a UI restriction — enforced identically on both sides. PO email and reporting are real; payments 'grouped by vendor/customer' is a real client-side grouping of real, permission-scoped payment records rather than a new backend aggregation endpoint.

A guided assistant that tracks human progress, never executes on its own

The AI Assistant's step-by-step guidance is deliberately split in two: a real Gemini-backed chat (with a Groq fallback and production-hardened timeouts, added after an observed queuing issue in production) that only ever relays pre-authored, human-written instructions verbatim — the model is structurally prevented from inventing step text. Progress through a workflow's steps is a separate, non-LLM state machine advanced by real signals: a real navigation event, a real UI success event from the action the step describes, or the user's own manual confirmation — never the model's own assertion that something is done. The guided-task tools can only ever write to their own bookkeeping record; creating a new business record (a contact, a deal, a purchase order) has no automated path at all — the assistant can only walk a person through doing it themselves, and any actual mutation it can propose still goes through the existing human-approval, 36-hour-delay pipeline.

> Every mechanism above is confirmed directly from source, including the parts a marketing description would be tempted to round up: static-rate currency conversion, browser-based PDF export, and an AI assistant that tracks rather than executes. Each area ships with its own test coverage.

SECTION 40

Real OAuth for Nine Business Integrations (September 2026)

The Integrations module's nine connectors (accounting, communications, storage, and productivity tools) moved from placeholder connect buttons to real OAuth 2.0 authorization-code flows, each provider using its own actual API — not a shared mock adapter behind nine different logos.

One real flow per provider, real tokens, real disconnect

Each integration completes a genuine OAuth handshake against that provider's own authorization server, stores the resulting access/refresh tokens per-org, and calls that provider's real API to confirm the connection actually works — not just that the redirect completed. Disabling or reconfiguring a connection revokes and clears the real stored credential rather than flipping a status flag, closing UX bugs where a disabled integration could still silently act as connected.

> Built and verified against each provider's real developer console in this pass, with the same 'no fake claims' discipline as the rest of the platform — a connection only shows Connected once a real token exchange and a real API confirmation call have both succeeded.

SECTION 41

Real-Time Voice Interface for the Business AI Assistant (September 2026)

A voice mode for the same Business AI Assistant described in §39 — speak a request, hear a spoken answer, with the assistant's existing tool-calling, permission-scoping, and Guarded Execution approval gate all unchanged underneath it.

Voice is a new input/output layer, not a new authority

Speech is transcribed to text and handled by the exact same business-chat pipeline and tool set already covering the typed assistant — the same permission scoping, the same inability to create records or bypass approval, the same 36-hour-delay pipeline for any proposed mutation. Nothing about arriving by voice grants the assistant a capability it doesn't have when typed.

> Shipped as an extension of an already-shipped, already-tested assistant rather than a parallel system — voice requests inherit every guardrail described in §39 without any separate code path that could diverge from them.

SECTION 42

Business Workspace UX/UI Makeover (September 2026)

A usability pass across the Business Workspace frontend — not a redesign. Three parallel research passes across all 36 view files found that the visual language was already a de facto consistent design system, hand-copied into existence dozens of times over rather than ever centralized: a Modal shell duplicated 12 times, status-color logic duplicated 6+ times, date/currency formatting hand-written 46 times across 19 files.

A real shared design system, extracted rather than redesigned

New shared components (Modal, StatusBadge, FormField, RecordRow, plus date/currency formatters) were extracted verbatim from the code already duplicated across the app, then verified to produce byte-identical visual output to what they replaced — confirmed live via computed-style comparison in the browser, not just visual inspection. Every migrated form gained a real, accessible label element; previously, none existed anywhere in the app.

Three real navigation bugs fixed

Two redundant 'home' screens were consolidated into one. Departments, Projects, and Documents all rewrote to one generically titled screen regardless of which was clicked — fixed to show the correct title. Two confusingly similar nav labels pointing at unrelated screens were renamed to describe what each screen actually shows.

> Jest and React Testing Library were introduced for the first time in this pass — previously zero frontend test tooling existed anywhere in the app. Migration covered the highest-traffic modules (Documents, Tasks, CRM, Procurement, Inventory, Finance, HR, Insights, Settings, Integrations); the remaining smaller views and one large outlier file were deliberately deferred and documented, not silently left inconsistent. No backend logic, permission check, or API contract changed anywhere in this pass.

SECTION 43

Multi-Cloud Storage Compatibility & Enterprise Storage Governance (September 2026)

Two passes, back to back. The first built real, protocol-level AWS S3 and Azure Blob Storage compatibility — verified against the actual AWS CLI and the official @azure/storage-blob SDK, not a simplified approximation. The second closed the gap between that and what enterprise storage buyers actually require: granular access grants, versioning, immutable retention, legal hold, lifecycle policies, and a real desktop drive mount. Both S3 and Azure objects run through the exact same real encrypt/shard/pin pipeline every other Inaya upload uses — this is a protocol translation layer, not a parallel storage system.

1. **Real SigV4 and Shared-Key request signing, implemented from the actual specs.** src/lib/s3-compat/sigv4.js (AWS Signature Version 4: canonical request → string-to-sign → derived signing key → HMAC) and azureAuth.js (Azure Shared Key). Both cross-checked line-by-line against the real SDKs' own bundled source, not just the public docs.
2. **Granular Storage Access Grants.** A credential can be scoped to one bucket, one prefix, a specific operation set (READ/WRITE/DELETE/LIST), and an expiry — enforced centrally in auth.js/azureAuthMiddleware.js after signature verification, deriving bucket/key/operation from the request's own URL rather than trusting anything the client claims.
3. **Object Versioning, Object Lock, Legal Hold — one shared enforcement chokepoint.** assertNotProtected() in store.js/walletStore.js is called from both the overwrite path and the delete path, so a lock or hold can never be bypassed by hitting the S3 API, the Azure API, or Business Workspace through a different route. Object Lock requires Versioning already Enabled — enforced, not just documented.
4. **Lifecycle policies, real enforcement.** Per-bucket expiration rules with a real enforcement pass (never expires a locked or legally-held object, even past its expiration day) — callable on demand or via the new /api/cron/s3-lifecycle route.
5. **A real, previously-unrecognized gap in storage health, closed.** S3/Azure-compat objects were pinned but never registered with the existing backupEngine.js health/repair pipeline (Section 27) — invisible to the check-pins/verify-integrity/recovery crons that already protect every other file. putS3Object now calls the same replicateShard() every other upload path calls.
6. **Inaya Drive — a real Windows drive letter, WinFSP-backed.** A standalone Rust process (inaya-drive-helper, its own Cargo crate) implements WinFSP's FileSystemContext trait, backed entirely by the same /api/s3 REST endpoint via a Rust port of the SigV4 signer. Kept as a separate process specifically because the Rust WinFSP binding is GPL-3.0 licensed — inaya-desktop spawns/kills it as a child process only, confirmed via a clean cargo check that zero GPL dependency reaches the proprietary desktop binary.

COMPONENT	DETAIL
S3 REST surface	src/app/api/s3/**/route.js — ListBuckets, bucket CRUD, ListObjectsV2, PutObject, GetObject (byte-range aware), HeadObject, DeleteObject, multipart upload, plus real sub-resources (?versioning, ?object-lock, ?versions, ?legal-hold, ?retention, ?versionId=).
Azure REST surface	src/app/api/azure/**/route.js — List Containers, container CRUD, List Blobs, Put/Get/Delete Blob (Range-aware), Put Block / Put Block List. Three ways to authenticate: Shared Key, time-limited SAS share links (added October 2026), or real Microsoft Entra ID Bearer tokens (verified live against Microsoft Graph).
Business Workspace console	src/app/api/orgs/s3-compat/manage/route.js + S3CompatView.js — bucket/version/lock/health visibility and controls, session-authenticated, separate from the SigV4-signed protocol surface.
Test coverage	19 new tests (test/s3-compat-capabilities.test.mjs) plus the pre-existing 9 (sigv4) + 9 (store) — all passing, zero regressions.

Real bugs found and fixed by real-tool testing, not code review. (1) A pin-name collision: the pinning-provider name passed to provider.pin() was identical across every version of the same key, and Filebase (this environment's active provider) uses that name as its literal S3 object key — two versions pinned under the same name silently overwrote the same provider-side object. Invisible before Versioning existed; caught by the first version-retrieval test. (2) putLifecyclePolicy/deleteLifecyclePolicy passed a bucket name string as the audit log's recordId, which requires a real ObjectId — threw on every write. (3) Inaya Drive: a read exactly at end-of-file issued an invalid Range request the server correctly rejected as 416, surfacing to Windows as "device not functioning." (4) Inaya Drive: delete silently failed because WinFSP's set_delete() callback — which must accept a delete before cleanup() can act on it — was never implemented. All four fixed and re-verified.

How this fits the zero-knowledge model in Section 05. Standard S3/Azure client tools don't perform Inaya's client-side encryption step — that's not a gap, it's how the AWS CLI and Azure SDK work for everyone, on any platform. So objects written through this compatibility layer specifically use a server-managed key, scoped per organization, itself encrypted at rest, with every use logged to the organization's own audit trail. A real, disclosed exception for this one access path only — every other Inaya upload keeps the client-side-only model unchanged.

Disclosed, not silently different (at the time of this pass): no presigned-URL SigV4 signing (only header-based); no empty-folder creation via Inaya Drive; Inaya Drive proven on Windows only; Object Mount packaged by running the compiled helper directly, not yet a proper Tauri externalBin sidecar. Every one of the first three items was closed in the follow-on work Section 44 below describes — kept here verbatim as the honest record of what was true at the time this section was written. Full writeup: docs/storj-inspired-storage-capability-expansion-report.md and docs/multi-cloud-storage-compatibility-report.md.

SECTION 44

Storage Interoperability Expansion — Empty Folders, Enterprise Migration Tooling & Google Cloud OAuth/Signed URLs (September 2026)

Three back-to-back passes that closed the disclosed gaps Section 43 listed and extended enterprise adoption further: Inaya Drive now supports real empty folders and a real Linux mount alongside Windows; a local, credential-never-leaves-your-machine migration agent moves existing AWS S3/Azure Blob/GCS data into Inaya; and the Google Cloud Storage compatibility layer gained OAuth identity federation, temporary signed URLs, and virtual-hosted bucket addressing.

1. **Real empty-folder support, on Windows and Linux both.** A new s3_folders collection (org side) plus direct reuse of the wallet's existing metadata_folders collection give folders a real, durable, non-versioned metadata row — never a fake `.folder`/`.keep` placeholder object, never touching encryption/sharding/pinning. inaya-drive-helper (Windows/WinFSP) now creates real folders instead of rejecting the request; a new inaya-drive-helper-linux (FUSE, MIT-licensed, no GPL obligation) does the same on Linux, sharing the identical S3 client/SigV4 signer via a new shared crate, inaya-drive-core, extracted specifically so neither helper duplicates the other's network code. Both live-tested including the strongest proof either SOW named: kill the mount helper process entirely, start a fresh one, confirm the empty folder survived.

2. **A local Data Migration Agent — credentials never leave the operator's machine.** inaya-migration-agent is a standalone Node CLI that reads from a real AWS S3, Azure Blob, or Google Cloud Storage source (via each provider's own real SDK) and writes through Inaya's existing, already-tested S3-compatible endpoint — no second encryption/sharding implementation. A resumable, append-only local manifest makes an interrupted run safe to simply re-run: already-migrated objects are skipped, never duplicated. Every migrated object is verified against the destination immediately after upload.

3. **Backup-tool and infrastructure-tool compatibility, proven against real, unmodified clients.** rclone and Terraform (the standard hashicorp/aws provider, pointed at Inaya's endpoint) both verified fully working end to end — real upload/download/sync for rclone, a real init → apply → plan → destroy lifecycle for Terraform, including force_destroy actually emptying a bucket. AzCopy was classified unsupported when this was first written, because it needs SAS-token authentication for Azure Blob. That gap was closed in October 2026: Azure share links (SAS) are now supported and were verified with the real AzCopy 10.32 (upload of a folder tree including a multi-block blob, list, download, remove, plus refusal of a read-only link used to write and of a tampered link). AzCopy treats the first part of a web address as the container, so listing and syncing folders needs a dedicated Azure hostname; that routing is built and off by default until an operator chooses a hostname and sets up its DNS and certificate (docs/azure-host-addressing.md). Running the real tools also found and fixed two genuine faults: concurrent first uploads into a new bucket could create duplicate buckets, and reading a just-replaced file could return the old copy (which made Terraform wait indefinitely).

4. **A Compliance Evidence Exporter, read-only by construction.** A new Business Workspace panel and API route aggregate an organization's own existing audit chain, bucket protection settings (versioning/Object Lock/lifecycle), and security event history into a downloadable JSON or PDF evidence package — with a real SHA-256 export hash over a canonical serialization, and a live re-verification of the audit chain's own tamper-evident integrity, not a cached flag. No new tracking collection, no new audit system: the one write this feature ever performs is logging its own generation event to the existing audit chain.

5. **Google OAuth, temporary signed URLs, and virtual-hosted bucket addressing.** A Google-verified identity (the same google-auth-library verification already used for the app's own Google sign-in) can now authenticate directly against the storage endpoint via Authorization: Bearer <ID token>, mapped to existing organization membership — no parallel permission system, mirroring the same pattern already proven for Microsoft Entra ID. A new ?presign sub-resource issues real, HMAC-signed, time-limited download URLs (an Inaya-specific scheme, disclosed as not byte-compatible with AWS/GCS's own presigned-URL algorithms) — proven with a genuine wall-clock expiration test, not a simulated clock. Virtual-hosted addressing (bucket-name-in-the-hostname) is implemented as a middleware URL rewrite onto the existing path-style routes, inert until an operator configures a wildcard host — no DNS/TLS change made automatically.

COMPONENT	DETAIL
New crate: inaya-drive-core	The Windows helper's S3 client and SigV4 signer, which had zero WinFSP dependency to begin with, extracted into a shared crate both the Windows and Linux Drive helpers link — no second implementation.
New: inaya-migration-agent	Standalone Node CLI (bin/inaya-migrate.mjs); AWS/Azure/GCS source adapters; Inaya destination via the real @aws-sdk/client-s3 pointed at Inaya's own SigV4-verified endpoint.
New: src/lib/s3-compat/signedUrl.js, src/middleware.js	Signed-URL creation/verification; virtual-hosted addressing rewrite (S3_COMPAT_VIRTUAL_HOST_BASE, unset by default).
New: src/lib/evidenceExporter.js, evidencePdf.js	Read-only evidence aggregation and PDF rendering (pdfkit — pure JS, no headless-browser dependency, so it runs in the real serverless deployment target).
Test coverage	17 new empty-folder tests, 9 signed-URL tests, 5 migration-manifest tests plus a 4-scenario live migration suite, 5 evidence-exporter tests, 5 Linux-helper unit tests — all passing, alongside the full pre-existing S3-compat regression suite re-confirmed clean throughout.

Real bugs found and fixed by real-tool testing, not code review — again. (1) The Windows Drive helper silently failed every mount with ERROR_DELAY_LOAD_FAILED because winfsp-x64.dll was never bundled next to the built exe — fixed via a build.rs step that copies it automatically. (2) read_directory's own listing prefix double-included the bucket name for any real nested folder, silently returning empty listings — found via live mount testing, fixed. (3) Next.js was issuing a 308 redirect for bucket-only requests ending without a trailing slash mismatch, which every real S3 SDK's CreateBucket/DeleteBucket call triggers — invisible until tested against genuine Terraform/AWS-SDK traffic, fixed via Next's own skipTrailingSlashRedirect flag. (4) Two missing real S3 sub-resources (bucket-wide ListObjectVersions, batch DeleteObjects, and separately GetObjectAcl) were found via live Terraform and gcloud storage testing specifically — each was a genuine gap blocking a real, unmodified client, not a hypothetical one.

gsutil and gcloud storage — an honest, corrected record. The earlier finding that gcloud storage doesn't support a custom S3-compatible endpoint was incomplete — it does, via gcloud config set storage/s3_endpoint_url, officially documented though Google itself labels it "unstable." Real upload, byte-identical download, and listing were proven working end to end against Inaya's live endpoint. gsutil's legacy boto2 stack still doesn't work: the previously-diagnosed region-detection crash is now confirmed avoidable with a real hostname containing "s3", but a separate connection-level issue remains in Google's own legacy code — documented as unresolved, with no third-party source patched, per this SOW's own explicit instruction.

Still disclosed, not silently different: the Inaya signed-URL scheme is not AWS SigV4/GCS V4 wire-compatible; macOS Drive support is written (the same fuser crate covers it) but not compiled or tested on real Mac hardware; file rename (as opposed to folder rename) has no backing primitive on either Drive helper yet; gsutil S3-compatible mode remains unverified for the reason above. Full writeup: docs/inaya-drive-empty-folder-creation-report.md, docs/enterprise-adoption-and-market-reach-report.md, and docs/gcs-compatibility-extension-report.md.

SECTION 45

AWS S3-Inspired Storage Feature Expansion (September 2026)

An audit-first pass against Amazon S3's own published feature catalogue — implementing only what the audit found genuinely missing and valuable, not cloning AWS's product. Object tags, a real independently-verifiable checksum, a storage inventory export, bulk batch operations, per-bucket analytics, and a read-only credential policy analyzer were added; event notifications/webhooks and MFA/step-up authentication were evaluated and explicitly deferred, with the reasoning documented rather than silently dropped.

COMPONENT	DETAIL
store.js/walletStore.js: putObjectTagging/getObjectTagging/deleteObjectTagging	Real S3 tag semantics (10-tag limit, 128/256-char key/value limits), wired into the object route's ?tagging sub-resource and the x-amz-tagging upload header.
store.js/walletStore.js: contentSha256 field	The real, unsalted SHA-256 of uploaded bytes — additive and distinct from the pre-existing fileHash field, which is deliberately salted for an unrelated legacy dedup reason and can't itself serve as a client-verifiable checksum. Exposed as x-amz-checksum-sha256.
New: src/lib/s3-compat/inventory.js	Org-scoped JSON/CSV export of every live object with its tags, checksum, and lock state — reusing existing records, no new tracking collection.
New: src/lib/s3-compat/batchOperations.js	Bulk tag/retention/legal-hold across up to 1,000 keys, built strictly as a loop over the existing per-object primitives — a locked object fails only its own key in the per-key result, Object Lock/Legal Hold is never bypassed.
New: src/lib/s3-compat/analytics.js, policyAnalyzer.js	Real per-bucket object count/size/version/lock counts and tag distribution; a read-only credential-scope analyzer flagging unrestricted/no-expiry/broad-destructive-scope credentials from real stored fields only — "unused credential" is honestly reported as not-computed, never fabricated, since no lastUsedAt field exists to support that claim.
Test coverage	9 new tests (test/s3-compat-expansion.test.mjs) plus the full pre-existing 57-test S3-compat regression suite re-confirmed clean, zero regressions.

A real, pre-existing bug found and fixed along the way. pinningProviders/pinata.js only implemented Pinata's newer JWT auth, while this environment's actual credential was stored as PINATA_API_KEY (a short key id) / PINATA_SECRET_API_KEY (which, despite its name, holds the real scoped-key JWT) — meaning Pinata had been silently unusable and the platform was falling back to Filebase as its de facto primary shard-storage provider instead of Filebase's intended backup-replication role. Fixed by detecting a JWT by its actual shape across all three candidate variables rather than one hardcoded name, and verified live with a real pin/status/fetch/unpin round trip.

Explicitly deferred, not silently skipped: event notifications/webhooks (deferred at the time, then built in October 2026: signed AWS-format webhook delivery for uploads, deletes and lifecycle expiry, with retry and backoff, a dead-letter list and manual redelivery, set up by an owner or admin in the S3 management screen); MFA/step-up authentication (Object Lock and Legal Hold already provide real, server-enforced protection against permanent deletion, the highest-risk case a step-up mechanism would otherwise protect); Block Public Access (confirmed not applicable — no code path anywhere in the S3-compat auth layer permits an unauthenticated read, so there is nothing to block). Full writeup: docs/aws-s3-feature-expansion-report.md.

SECTION 46

Evidence Graph & Digital Twin — Trusted Business Events and What-If Simulation (September 2026)

Two connected SOWs: the Evidence Graph links an organization's existing invoices, purchase orders, purchase requests, and AI-proposed actions into one traceable Business Event with a portable cryptographic-proof passport and a read-only What If simulator; the Digital Twin extends that same read-only simulation discipline into a broader dependency graph across the organization's real entity landscape, answering "what would happen if a supplier became unavailable, an employee lost project access, a project was delayed, or a warehouse went offline."

COMPONENT	DETAIL
New: src/lib/businessEvents.js	The Business Event core — references a subject by {subjectType, subjectId} rather than copying it, department/org-manager scoped exactly like every other Business Operations record. Every mutation flows through the existing logOrgActivity(), which already writes into the real cryptographic audit chain — no second audit system.
New: src/lib/businessEventExplain.js	The Why? view — permission-aware evidence resolution (RESTRICTED/INCLUDED/REFERENCED/UNAVAILABLE), AI findings limited to real stored fields, never internal model chain-of-thought, because none is persisted anywhere to begin with.
New: src/lib/businessEventPassport.js	A JSON+PDF Business Event Passport reusing the existing Compliance Evidence Exporter's exact canonicalize/hash convention, with independent VERIFIED/INVALID/INCOMPLETE/UNKNOWN re-verification.
New: src/lib/businessEventSimulate.js	Read-only What If simulation — re-derives transition legality/authorization from the real state tables and permission gates without ever importing the real transition functions.
New: src/lib/digitalTwin.js, digitalTwinSimulate.js	A dependency graph computed on demand from real, existing foreign keys (a supplier's own POs, a project's own tasks, a person's own task assignments) — no new graph database. Four named scenarios (SUPPLIER_UNAVAILABLE, EMPLOYEE_ACCESS_REMOVED, PROJECT_DELAYED, WAREHOUSE_UNAVAILABLE), each reporting a numeric consequence only when a real stored field backs it, UNKNOWN otherwise.
Test coverage	47 tests for the Evidence Graph layer, 9 for the Digital Twin layer, all passing against the real database. The strongest test in both: run every simulation type, snapshot every touched record before/after, assert byte-for-byte equality — a simulation can never silently become a real mutation.

A real permission gap found and fixed during development. stockLevels rows carry no departmentId of their own (see inventory.js) — the generic per-dependent permission check that correctly gates every other entity type would have silently defaulted a stock-level row to visible regardless of the caller's actual department access. Fixed by gating the WAREHOUSE_UNAVAILABLE scenario on the warehouse's own department before any traversal begins, and covered by a dedicated security test.

Cross-organization Digital Twins (two companies privately checking each other's fulfilment capacity without exposing internal data) were evaluated, not built: zero-knowledge proofs, secure multiparty computation, and trusted-execution options were all assessed and the recommendation is to defer, since no validated customer need exists yet and building that cryptography speculatively isn't justified. Graph-visualization UI, versioned Twin snapshots, and natural-language scenario creation are likewise deferred until the underlying model has real usage to build on. Full writeup: docs/evidence-graph-business-event-layer-report.md and docs/digital-twin-simulation-layer-report.md, docs/digital-twin-cross-org-privacy-feasibility.md.

SECTION

Modular Enterprise Adoption Layer — DirectSync, Data Room Templates, Cloud Backup Scheduler & What-If Studio (September 2026)

Four SOW features, each preceded by a mandatory capability audit (published as docs/MODULAR_ADOPTION_CAPABILITY_AUDIT.md) classifying every proposed capability as already-implemented, reusable, a genuine gap, or an architectural decision requiring sign-off — no feature here duplicates an existing system. DirectSync is a native background local-folder watcher; the other three extend existing Data Room, migration, and Digital Twin infrastructure rather than sitting beside it.

COMPONENT	DETAIL
New: inaya-desktop/src-tauri/src/directsync.rs	DirectSync's engine — SQLite-backed local state (folders/sync_state tables), SHA-256 change detection (never trusts mtime alone), notify-debouncer-full for coalescing duplicate OS filesystem events and pairing rename from/to paths, and an upload path through inaya-drive-core's own S3Client/SigV4 signer — the same client inaya-drive-helper and inaya-drive-helper-linux already use against the real /api/s3 endpoint. A local delete only marks the row LOCALLY_DELETED; the remote object is never touched — backup semantics, not mirror semantics, by design.
Modified: inaya-desktop/src-tauri/src/lib.rs	10 new Tauri commands (directsync_add_folder, _remove_folder, _pause_folder, _resume_folder, _list_folders, _list_queue, _retry_failed, _store_credential, _credential_configured, _clear_credential, _pick_folder), all gated by the file's existing verify_trusted_origin defense-in-depth idiom. DirectSyncState is initialized in .setup() — restoring the credential from the OS keychain (via the same keyring crate already used for the master-node passkey) and resuming every enabled folder's watcher — and torn down on tray Quit, mirroring the existing DriveState pattern.
New: src/lib/dataRoomTemplates.js	Extends external-data-room.js (not a fork) with four built-in templates (Fundraising, M&A, Legal Review, Web3 Due-Diligence), org-owned custom templates, NDA gating, and section-tagged document uploads. createRoomFromTemplate() bridges into the existing createDataRoom().
New: src/lib/dataRoomEvidence.js	Verified evidence export for a data room, reusing evidenceExporter.js's exact canonicalizeForExport/sha256 convention — the same technique businessEventPassport.js already established.
New: src/lib/cloudBackupScheduler.js	Orchestrates inaya-migration-agent's real, already-tested source adapters and runMigration() engine — linked via a file: dependency, not rewritten. Resolves two real semantic gaps in that package (a local-file Manifest incompatible with serverless invocations; permanent-once-migrated semantics wrong for a recurring backup) with its own Mongo-backed incremental diff (backupObjectState) and a duck-typed shim manifest, without modifying the shared package. Destination writes go through s3-compat/store.js directly, in-process, rather than round-tripping through the package's own HTTP-oriented client.

COMPONENT	DETAIL
New: src/lib/backupCryptoAndCredentials.js	Envelope-encrypted (AES-256-GCM) storage of a customer's own AWS/Azure/GCS credentials, under a key dedicated solely to this feature — a disclosed, deliberate departure from the migration tool's own "credentials never leave your machine" design, necessary for anything to re-authenticate on a recurring schedule.
Modified: src/lib/digitalTwinSimulate.js	Added an independently-recomputable integrityHash (reusing evidenceExporter.js's canonicalization), simulationId, and a new listDigitalTwinSimulations() for scenario history — the What-If Studio UI's only new backend surface; the simulation engine itself is unchanged.
New: src/components/business/WhatIfStudioView.js, DataRoomsView.js extensions, CloudBackupSchedulerView.js, DirectSyncView.js	Business Workspace UI for all four features, following the existing self-contained-view convention. DirectSyncView calls native Tauri commands via window.__TAURI__.core.invoke() (the same pattern the app's existing pending-approvals/security-feed pollers use) and degrades to an honest "requires the desktop app" message in a plain browser tab.
Test coverage	Data Room Templates: 8 tests, zero regressions to external-data-room.test.mjs. Cloud Backup Scheduler: 12 tests, including a genuine end-to-end run through the real migration engine and real storage write path via node:test's mock.module() substituting only the external cloud source adapter. What-If Studio: 2 additional tests on the existing Digital Twin suite. DirectSync: 7 Rust unit tests plus 1 real, non-mocked #[ignore] end-to-end test run manually against a live dev server and a real, disposable S3-compat credential (read from environment variables, never hardcoded) — proving real upload+verify, duplicate-safety, change detection, rename with actual remote relocation, and delete-preserves-remote.

Two real bugs found and fixed during development. cloudBackupScheduler.js's first version computed both objectsSeen and objectsChanged from the same value, so a no-op run's "seen" count silently equaled its "changed" count instead of reflecting every object actually examined — caught by a duplicate-safety test, fixed by tracking the diff's own seenKeys.size separately. directsync.rs's first version of handle_rename() only updated local SQLite bookkeeping without actually relocating the object in remote storage — caught by the real end-to-end test (a HEAD on the renamed key returned nothing), fixed by having the rename path PUT the unchanged local bytes under the new key and best-effort DELETE the old one.

Built later (October 2026): DirectSync Share, a time-limited download link for any synced file, signed on the machine as a standard presigned URL and verified by the server. Deliberately not built in this pass: a real-action handoff from a What-If simulation into the existing PENDING_APPROVAL flow (no concrete use case has proposed one yet), and DirectSync running independent of the desktop app being open at all (it runs as a background task inside inaya-desktop's existing process, not a fifth standalone native service — a deliberate reuse of real tray/background infrastructure over building a new OS-level service installer). Full writeup: docs/directsync-report.md, docs/cloud-backup-scheduler-report.md, and docs/MODULAR_ADOPTION_CAPABILITY_AUDIT.md.

SECTION

IBM Cloud VPC Storage-Inspired Capability Expansion & a Real Terraform Provider (September 2026)

A mandatory Phase 0 audit (docs/IBM_VPC_STORAGE_CAPABILITY_AUDIT.md) found the one fact that shaped every design decision below: Inaya has no compute/VM/hypervisor runtime of any kind. IBM VPC's block-volume model assumes a running instance to attach a device to; Inaya has nothing of the sort. Rather than fabricate a fake compute layer or quietly skip the workstream, every "physical infrastructure" capability was rebuilt as an honest, useful, logical control-plane abstraction, labeled as exactly that. Once real Go tooling was set up in this environment, the SOW's own explicitly-deferred Terraform workstream was also built and tested end-to-end.

COMPONENT	DETAIL
New: src/lib/storageResources.js	The Storage Resource Registry — no generic resource envelope existed anywhere in this codebase before this SOW. createStorageResource/listStorageResources/getStorageResource/expandStorageResourceCapacity/deleteStorageResource, every resource backed by a real S3-compat bucket. attachVolume/detachVolume is a real reservation/lock (attachedTo is a caller-supplied free-text label, never a fabricated device path); addMountTarget/removeMountTarget on a file share are declared, bookkeeping-only records (physicallyMountable: false). Both resource types carry a static physicalCapability field that is never upgraded to imply real capability. Capacity only ever increases, per the SOW's own explicit rule.
New: src/lib/storageSnapshots.js	Built entirely on s3-compat/store.js's existing per-key S3 Versioning — creating a snapshot enables bucket Versioning and records a manifest of every current key's versionId; no bytes are copied at capture time, making the "incremental" label genuinely true rather than asserted. restoreSnapshot() is a real copy-forward restore. Consistency groups capture member snapshots sequentially and honestly carry consistencyBoundary: "SEQUENTIAL_NOT_ATOMIC" rather than a false claim of simultaneous crash consistency. Cross-org sharing (shareSnapshot/revokeSnapshotGrant/resolveSnapshotGrant) fails closed on wrong recipient, revocation, or expiry.
New: src/lib/storageBackupPolicies.js	The Inaya-native complement to cloudBackupScheduler.js (which pulls FROM an external cloud source INTO one hardcoded bucket) — this policy-selects the org's OWN storageResources.js resources by tag and recurringly snapshots them, with real multi-plan-per-policy (daily/weekly/monthly/longTerm) and real retention enforcement (oldest snapshots beyond a plan's retentionCount deleted, every deletion audited via logOrgActivity). Health status mirrors cloudBackupScheduler.js's own HEALTH_STATUS thresholds exactly.
16 new API routes + 1 cron route	src/app/api/orgs/storage/{resources,snapshots,consistency-groups,backup-policies,backup-plans,backup-jobs}/**/route.js (session-cookie auth, requireMembership) plus src/app/api/cron/storage-backup-run/route.js (vercel.json, hourly).
New: src/app/api/public/v1/storage/** (added with the Terraform provider)	A parallel, bearer-API-key-authenticated namespace (requireApiKey, same resolution guarantee as the existing public/v1/evidence route — orgId always comes from the key, never the request body). Built specifically because Terraform runs headless and can't use requireMembership()'s session-cookie resolution. Backed by two small additive lib functions the schema already anticipated: deleteBackupPolicy/deleteBackupPlan soft-delete via a deletedAt field that existed on both collections from day one with nothing that ever set it — findDueBackupPlans() and runBackupPolicyPlan() already filtered on it.
New: terraform-provider-inaya/ (Go, terraform-plugin-framework)	inaya_storage_resource, inaya_snapshot, inaya_backup_policy, inaya_backup_plan resources against a hand-rolled ~250-line HTTP client (internal/client/client.go) — no generated SDK, the surface is 8 routes. Every attribute without a real backend update path (name, type, region, tags on a resource; name/tag_selector on a policy; every field on a plan) is RequiresReplace rather than silently no-opping a change the backend can't actually make; capacity_gb is the one real in-place Update, and a decrease is surfaced as a Terraform-level error, not silently ignored.
Test coverage	35 automated tests across storageResources.js, storageSnapshots.js, storageBackupPolicies.js, and the Digital Twin integration, all passing against the real database and real S3-compat write path (6 pre-existing, unrelated tests in the same run failed on an external Pinata account-quota block, not on anything in this SOW). The Terraform provider's full create/read/update/delete cycle — including cross-resource references, a no-op plan showing zero drift, and an in-place capacity expand — was run against a real local Inaya dev server with real MongoDB-backed state, not a dry run.

The STORAGE_RESOURCE Digital Twin permission gate is a bespoke special case, not the generic table entry. digitalTwin.js's resolveDependents() gained a dedicated STORAGE_RESOURCE block (imports canAccessStorage) rather than a DEPENDENT_RESOLVERS table entry, because storage resources are org-wide — gated by the distinct canAccessStorage permission, not department membership. The generic table's own "no departmentId means visible to everyone" default would have been a real permission/data leak for this entity type.

A real bug found during the Terraform provider's own end-to-end testing. createBackupPlan()'s and createSnapshot()'s own return values omit fields only the list/get functions compute or attach (health, createdAt) — so inaya_backup_plan.health and inaya_snapshot.created_at came back blank immediately after terraform apply, correcting themselves only on the next refresh. Fixed by having every resource's Create() re-fetch via the corresponding Get call before writing Terraform state, for all four resource types.

Deliberately not built: fast/accelerated restore (no backend primitive exists to make one honestly faster than the normal copy-forward restore above — never labeled "fast" to check a box), real physical block-volume attach or real multi-client NFS mounting (structurally impossible without a compute layer that doesn't exist), context-based/conditional access restrictions (no concrete use case identified yet beyond the existing role/department/assignment model), and live interop validation against real IBM Cloud Object Storage (blocked on IBM Cloud not accepting the available payment method during account sign-up, not on anything left to build — the exact ibmcloud CLI commands to obtain the needed instance/HMAC credentials/endpoint are documented and ready once an account exists). The Terraform provider itself is not yet published to the Terraform Registry — it runs from a local build via dev_overrides today. Full writeup: docs/ibm-vpc-storage-expansion-report.md, docs/IBM_VPC_STORAGE_CAPABILITY_AUDIT.md, and terraform-provider-inaya/README.md.

SECTION

Official Documentation Platform (IBM Cloud Docs-Inspired) — inayanetwork.com/docs (September 2026)

A mandatory Phase 0 audit (docs/audit/documentation-inventory.md) confirmed no /docs, /help, or KB route existed anywhere in this codebase, no OpenAPI spec existed, and no MDX tooling was in use — but a real, production RAG pipeline already grounded three AI assistants. This SOW builds a real documentation platform on that foundation rather than a second AI stack: Phases 0–3 plus slices of 4–7 of the SOW's own 15-phase plan.

COMPONENT	DETAIL
New: content/docs/**/*.md + src/lib/docsContent.js	A new content directory, deliberately separate from the internal, SOW-report-only docs/ directory (every file sampled there self-identifies as an internal architecture doc, none RAG-ingested, none public). The loader (gray-matter, the one new small dependency) validates every required frontmatter field and rejects a duplicate slug by throwing, not by best-effort rendering; a table of contents is derived from real ## / ### headings, never hand-maintained separately.
New: src/app/docs/** (Next.js App Router), src/components/docs/**	Homepage (real client-side search + four working destination cards, no card linking to an empty page), product guide pages, developer pages, and a DocsShell client component scoping the light/dark theme toggle to a local wrapper div rather than the document root, so it can never affect any other page on the site (tailwind.config.js's darkMode:"class" was added, previously unset).
New: src/lib/docsApiReference.js, docsSdkReference.js, docsCliReference.js	Hand-authored directly from the Phase 0 audit — all 11 real /api/public/v1/** routes, all 5 published npm packages, and all 3 published CLI tools' real commands. No OpenAPI spec or route-annotation system existed to generate from, so this reference data is the authoritative source for both the rendered pages and, as of this pass, the generated OpenAPI spec.
New: scripts/docs/generate-openapi.mjs → public/openapi.json	Generates a real OpenAPI 3.0.3 document directly from docsApiReference.js (path parameters auto-derived from each route's own {placeholder} syntax, query/body parameters mapped from the existing reference data). Validated with @apidevtools/swagger-parser; a test asserts the spec's paths/operations/security/path-params can never drift from the reference data the rendered pages use.
New: src/app/docs/release-notes/page.js	Renders directly from src/lib/saasRoadmap.js's existing ROADMAP_STAGES rather than a second, hand-authored changelog — the same source of truth /business/roadmap itself renders from, so the two can never disagree.
New: content/docs/developer/contributing.md	Honestly documents that no formal CI/review gate exists yet for this content, rather than implying one does.
Reused, not rebuilt: src/lib/rag/sources/docsSources.js	Gained one new docs-platform:* source per content/docs/**/*.md file, the exact same shape as the 15 existing fundraising:* sources — the existing Docs AI Assistant (POST /api/ai/chat) now retrieves from this new content on its next reingest cycle, without a second AI stack.
Test coverage	10 tests total: docs-content.test.mjs (9 — unique slugs, required-field/status validation, relatedDocs cross-reference integrity, heading extraction, search-index shape, product grouping) and docs-openapi.test.mjs (1 — the spec-matches-reference-data regression guard). A clean npm run build with every /docs route statically generated, plus real browser verification (including a genuine mobile bug found and fixed — a search button overlapping its own placeholder text at narrow widths).

An unrelated, pre-existing production bug found and fixed along the way. Shipping this surfaced that Vercel deployments had been failing silently for roughly 20 hours (confirmed via `vercel ls`/`vercel inspect --logs`) — cloudBackupScheduler.js imports ../inaya-migration-agent, a file: dependency, but Vercel only ever ran `npm install` inside inaya-network-dapp, never inside that sibling package, so its own @aws-sdk/client-s3 / @azure/storage-blob dependencies were never present in that environment (it worked locally only because that directory's node_modules had been populated by hand at some point). Fixed with a postinstall hook, verified by moving the local node_modules aside to simulate a fresh clone before shipping — not just assumed fixed.

Deliberately not built this pass: a Tutorials/Solutions/FAQ-as-a-system content type (the schema/renderer support them; no content exists yet), API/SDK/CLI drift checking against the live route files (the reference data is accurate today but nothing detects future divergence), a full CI validation/accessibility/SEO suite, an admin/governance interface, an API playground, and the docs.inayanetwork.com subdomain (ships at /docs on the existing deployment instead, which the SOW itself names as an acceptable fallback). Full writeup: docs/official-documentation-platform-report.md, docs/audit/documentation-inventory.md, and docs/architecture/information-architecture.md.

SECTION 47

Mainframe & Legacy Data Access — SQL Virtualization Layer (September 2026)

A live SQL virtualization layer sitting in front of external relational data sources: connect, publish a virtual schema, query with standard SQL — the data never leaves its source system.

LAYER	WHAT IT DOES
Connector registry	Pluggable interface (isConfigured/testConnection/discoverMetadata/executeQuery/health/capabilities); credential envelope encryption per data source.
Relational reference connector	Built on node:sqlite; real connection, metadata discovery via sqlite_master/PRAGMA, real SELECT execution incl. WHERE/JOIN/GROUP BY/aggregates.
Metadata & virtual schema engine	Schema discovery with explicit, versioned publishing — a re-import creates a new version rather than overwriting.
SQL gateway	Real SQL AST parser (node-sql-parser), authorization against published tables only, row/timeout-limited execution, every query (executed/denied/failed) audited via logOrgActivity.
REST API	/api/orgs/data-sources/** (session-auth) and /api/public/v1/data-sources/** (API-key-auth) — both backed by the same library code.
JDBC driver	jdbc-driver/ — a real, compiled java.sql.Driver, Maven-shaded jar.
ODBC driver	odbc-driver/inayaodbc.dll — a real, compiled Win32 DLL exporting 34 standard ODBC entry points, built with MinGW-w64 GCC against the real ODBC SDK. Registration and connection through the real Windows ODBC Driver Manager (the path Excel/Power BI use) are now proven — five real driver bugs were found and fixed to get there.
RMS/OpenVMS connector	connectors/rmsOpenVms.js — SSH + real DCL against a genuine VSI OpenVMS x86-64 instance. Metadata parsed from OpenVMS's own FDL output. 7/7 tests passing against the live instance.

Known gaps, stated plainly. Adabas, VSAM and IMS connectors are not implemented (Adabas has a real, healthy test environment running in Docker; VSAM/IMS require a genuine z/OS environment). The RMS/OpenVMS connector's fixed-format binary and indexed files aren't supported yet, pending a compiled OpenVMS-side reader. Write-back (INSERT/UPDATE/DELETE) and federated cross-source joins are not built this pass. ODBC query execution specifically through the Windows Driver Manager still crashes — isolated to a fault inside Microsoft's own odbc32.dll, not this driver.

SECTION 48

Sovereign NAS — Appliance and Control Plane (September 2026)

A control plane in the main app, an appliance agent on the Linux box, and 49 API routes and an 18-section console connecting them — none of it duplicating Inaya's existing identity, audit, Evidence Graph or Digital Twin layers.

LAYER	WHAT IT DOES
Appliance	Ubuntu 26.04 (WSL2 VM in the tested profile): Samba 4.23, nfs-kernel-server (NFSv4.2), mdadm RAID1, Btrfs, ext4 quotas, POSIX ACLs, chattr immutability.
Appliance agent (src/lib/nas/appliance/inaya-nas-agent.py)	One validated JSON request per operation, argument-list commands only (no shell strings), path containment, hash-pinned versioned install with rollback.
Control plane (src/lib/nas/*, 25 modules)	49 API route files under /api/orgs/nas, an 18-section console, scripts/nas-worker.mjs / /api/cron/nas.
Reuse	Org identity/permissions (orgGates), the cryptographic audit chain, Evidence Graph, Digital Twin engine, evidence exporter, s3-compat encrypted storage, notifications — none duplicated.

Boundaries, stated plainly. Physical hardware deployment is not done — the profile is a VM with virtual disks. NAS-to-NAS replication was tested with both appliances on the same host; cross-host transport is not implemented. Active Directory/LDAP, iSCSI, a local S3 gateway and Kubernetes CSI are not implemented. macOS clients are untested.

SECTION 49

AI Security Workflow — Gateway Architecture (September 2026)

Two integration points, not one all-in-one middleware: checkInputSecurity() runs after identity/authorization (already resolved by the caller) and before the model call; validateOutput() runs after the model produces text and before it reaches the user. A route's own retry/timeout/fallback logic is untouched.

MODULE	ROLE
src/lib/aiSecurity/promptInjection.js	Regex-pattern detection across 5 attack families (instruction override, system-prompt extraction, role manipulation, authorization spoofing, policy bypass); wrapUntrustedContent() for prompt-shielding retrieved content.
piiDetector.js	Email/phone/SSN/Luhn-validated card detection with in-place redaction.
policyEngine.js	Deterministic SecurityDecision evaluation (input, retrieved-content, output, action).
orgPolicy.js	Versioned per-org policy, manager-only writes, every version retained.
modelRegistry.js	Component inventory, seeded from what is actually configured; checkModelIntegrity.
events.js	AISecurityEvent recording into the real audit chain and Evidence Graph (new subject type AI_SECURITY_CHECK).
gateway.js / routeGuard.js	checkInputSecurity()/validateOutput(), the two integration points; guardAiInput/guardAiOutput wired into all 6 text routes.

Boundaries, stated plainly. Voice (voice-session, voice-tool-relay) is not gateway-covered — speech goes browser to Gemini Live directly, so there is no server-side text to inspect. Org-less surfaces (wallet, security, Learn assistants, docs chat) run the platform default policy and log with orgId: null, since there is no organization to own an audit-chain entry. Supply-chain dependency scanning and model-configuration drift monitoring beyond the static registry check are not addressed this pass.

SECTION 50

Document & Invoice Automation — Pipeline Architecture (September 2026)

data -> template -> calculation -> document -> validation -> approval -> evidence -> encrypted storage -> secure delivery -> verification. Nothing in the pipeline branches on a document-type name — a registry of adapters, templates, validators and policies serves all nine types.

LAYER	WHAT IT DOES
Adapters (src/lib/documentAutomation/adapters.js)	Read-only adapters onto Finance invoices, CRM contacts/deals, Procurement POs and payments — the source of truth for every document.
Calculation (money.js, calculations.js)	Exact currency-exponent decimal parsing, per-line tax/discount/shipping, pro-rata allocation, deterministic across re-renders.
Rendering (renderer.js)	pdfkit, bundled Noto Sans + Noto Sans Arabic (OFL); bounded to 200 pages / 20 seconds.
Storage (s3-compat/store.js)	Server-managed AES-256-GCM, sharded, pinned; per-org bucket generated-documents-<orgId>, Versioning + Object Lock.
Lifecycle (lifecycle.js)	DRAFT → GENERATED → PENDING_APPROVAL → APPROVED → FINALIZED → DELIVERED → VIEWED/PAID; pipelineState tracks GENERATING/STORAGE_FAILED/EVIDENCE_PENDING/COMPLETE separately from business status.
Evidence	New subject type GENERATED_DOCUMENT; documents link SOURCED_FROM / DERIVED_FROM / PROVEN_BY into the existing Evidence Graph and audit chain.

Production defect found by the live check, and fixed (SQA-026). pdfkit was bundled into the server chunks but loads its built-in fonts through a build-time absolute path and a package `imports` alias the file tracer cannot follow, so the font data was never shipped to the Vercel function — every PDF export failed, including the Business Event Passport. Fixed by keeping pdfkit external and explicitly tracing its font data (including a shared chunks/ helper) for every API route. Verified in a simulated function holding only the shipped files, then confirmed live in English, Arabic and Urdu on 2026-09-27.

SECTION 51

AI Business Operations Manager — Workflow Engine Architecture (September 2026)

A durable, permission-scoped workflow engine reusing the existing cron pattern (no second scheduler), the existing Controlled Actions approval flow (no second approval engine), and the existing notification/audit/Evidence Graph systems.

LAYER	WHAT IT DOES
Editor & validation	Visual node canvas; draft/publish/version/rollback; live validation before publish.
Triggers	Vercel cron (paid plan, runs as often as every 5 minutes) for schedule/webhook/API-key/event triggers.
Nodes (src/lib/workflows/nodes.js)	Data readers, transforms (merge/join/filter/map/select/rename/sort/aggregate/group/dedupe/derive — no code step), an AI-agent node, rule conditions, notification and approval nodes.
Engine (src/lib/workflows/engine.js)	Permissions follow the person RUNNING the workflow, not the author; a run resumes safely after a crash; nothing is sent or requested twice on retry.
AI Operations Manager	Gemini, server-side key only, returns a structured urgent/classification/confidence/findings/recommendations assessment; tool names, arguments and permissions are checked server-side, never trusted from the model.
Evidence	Every run recorded in the audit chain and Evidence Graph with an exportable Evidence Passport and a plain-language explanation of each decision.

Real bugs found and fixed this pass. The Business Brief node used the wrong period names (week instead of weekly); the executions list showed "0 nodes"; the passport route revealed that an execution existed to someone with no access to it; the AI step could only read its immediately preceding step, now fixed to read any completed step.

SECTION 52

Customer Portal & Customer Service — Architecture (September 2026)

Ticketing did not exist in Inaya before this work; it is now the core of a support module reusing, not duplicating, existing identity, storage, audit and AI-security infrastructure.

LAYER	WHAT IT DOES
Domain libraries (src/lib/support/*.js, 30 collections)	db, settings, access, sla, tickets, messages, queues, customers, attachments, portalAuth, inbound, kb, ideas, csat, ai, analytics, webhooks, apiKeys, incidents, macros, exporter, retention.
Dispatchers	agentApi.js (console/admin), portalApi.js (customers), publicApi.js (API keys).
Agent console	src/components/business/SupportView.js — queues, assignment, SLAs, macros, internal notes.
Customer portal	src/app/portal/[slug] — responsive, keyboard- and screen-reader-friendly, one-time-link or OpenID Connect sign-in.
Help & Support (new)	src/lib/help.js — Inaya's own users file a ticket into a designated organization's support desk (INAYA_SUPPORT_ORG_ID), from the verified session email, with an optional email copy to a support mailbox (INAYA_SUPPORT_NOTIFY_EMAIL, Reply-To set to the requester).
Reuse	Customers are crm_contacts; invoices read from invoices (never copied); files use the S3-compatible encrypted store with provider fallback; audit uses logOrgActivity; the Evidence Graph gets a SUPPORT_TICKET subject; AI goes through the AI Security gateway.

Boundaries, stated plainly. Inbound email (Resend adapter, Svix signature verification) is built but not yet live — its provider setup is pending. Company single sign-on and the malware scanner are implemented but not yet verified against a real identity provider or a real infected file in production.

SECTION 53

Identity Integration — Architecture (September 2026)

Identity is keyed by (provider tenant, immutable object id); email is only a controlled fallback with ambiguity failing closed. One tenant belongs to exactly one organization.

LAYER	WHAT IT DOES
providers.js, mapping.js	Identity/tenant mapping, versioned group and attribute mapping — owner never grantable by a directory rule.
engine.js	Joiner/mover: plan → execute → verify, preserving manual, existing and other-source access.
overrides.js, grants.js	Manual override ledger, reason required, labelled INAYA MANUAL OVERRIDE.
revocation.js	Leaver: freeze, sessions, credentials, permissions, sharing, break-glass — each independently verified, PENDING/PARTIAL/COMPLETE/FAILED, retry only unverified steps.
scim.js, /api/scim/v2	Standard SCIM 2.0 Users/Groups, discovery, filters, PATCH; every write is a lifecycle event.
api.js, outbound.js	Automation-platform actions and outbound events (Rewst-compatible); webhook route with HTTPS, signature, timestamp, replay, size, tenant, schema, idempotency, rate limit.
reconcile.js	MATCH/DRIFT/CONFLICT/UNRESOLVED, chunked snapshots, reports plus optional remediation.
msp.js, credentials.js	MSP delegation — two-sided links, four roles, re-verified per request; credential lifecycle (create/scope/rotate/revoke/expire), hashes only.
ad-sync-agent/	A real, standalone on-prem agent a customer runs next to their own domain controller — outbound-only LDAP to the DC, HTTPS to Inaya, never the reverse. Real full and incremental sync via AD's own uSNChanged watermark, feeding the same signed webhook and engine above unmodified.

Boundaries, stated plainly. Group-membership push from Entra has not been observed. Rewst is built to its open contract but unverified against the real product. Active Directory is now verified for real: a genuine Windows Server domain controller was stood up, a real test user (mail/department/title) was created in it, and the sync agent pulled and pushed her through the unmodified engine end to end. Okta, RMM, PSA and HR adapters remain generic and unverified against a real vendor.

SECTION 54

AI Bookkeeper — Architecture (September 2026)

Bookkeeper tables hold proposals and evidence; invoices, expenses, payments and purchase orders stay authoritative. An AUTO decision is internal state only — an authoritative change always needs a person.

LAYER	WHAT IT DOES
sources.js, bank.js	Encrypted source secrets; CSV/OFX/QFX parsing with occurrence-aware fingerprints for duplicate safety; a provider adapter interface with no live provider registered.
extract.js	unpdf text extraction, deterministic field extraction with line/snippet provenance, arithmetic validation, an AI-merge path grounded so the AI cannot invent a value not present in the source text.
categorize.js	Rule > approved mapping > history > AI, versioned and audited rules.
match.js, reconcile.js	Combined/partial/over/fee/currency matching; a three-way match over real PO items and received quantities; idempotent reconciliation runs.
policy.js	Four confidence dimensions (extraction/categorization/match/anomaly), configurable thresholds, risk overrides confidence.
review.js	Ten review actions, permission-gated, audited, feeding back into learned mappings.
Posting boundary (reconcile.js)	A confirmed match records a payment and, for a supplier bill, a DRAFT expense; "mark paid" is only ever PROPOSED through the existing Controlled Actions.

Boundaries, stated plainly. No live bank feed provider is registered (statement import only). Live email and WhatsApp are implemented against a signed relay and a stand-in for Meta's Graph API respectively, not against real accounts. OCR of images and scanned PDFs has no local engine and is never auto-processed.

SECTION 55

Whole-Codebase Quality & Security Review — Method and Findings (September 2026, in progress)

Discover -> baseline -> map -> test -> reproduce -> classify -> fix -> regression test -> cross-repository verify -> secure -> prove -> document. Each confirmed defect gets an id, severity, root cause, fix and a regression test that fails on the old behaviour.

SEVERITY	EXAMPLE FINDING	FIX
S0 (critical)	The bridge relayer would sign any registered message; a forged transfer could have been approved and executed.	The relayer now verifies the source chain's own MessageSent event before signing; the hash formula is pinned against the real contract.
S1	A retried Stripe webhook could settle a corporate payment on-chain twice.	Settlement is claimed atomically per session, with per-step progress so a retry resumes rather than repeats.
S1	Object keys with spaces, brackets or non-ASCII characters could not authenticate against Inaya's S3-compatible endpoint.	The SigV4 canonical-URI encoding bug is fixed; proven with the real AWS CLI against 9 awkward key names.
S1	Every PDF export failed in production (font data not shipped to the serverless function).	pdfkit kept external, font data traced explicitly for every route; confirmed live in English, Arabic and Urdu.
S2	Parallel faucet requests could exceed the lifetime token cap.	Atomic reservation before sending, released on failure.

Coverage, stated plainly. 225 non-organization routes and 453 organization routes are swept automatically on every run. The main dApp's contract test suite (175 tests) was fixed and now runs under the standard command. Not yet done: Azure Blob, Google Cloud Storage and Terraform against Inaya storage with their real client tools; physical hardware for mobile/desktop/Inaya Drive; concurrency/failure injection; smart-contract static analysis.

SECTION 56

Managed Database Service — Architecture (September 2026)

Vercel serverless functions cannot host a persistent database engine process (no listening TCP port survives between invocations, no container/VM layer exists in this stack). Workstream A therefore provisions and controls a REAL database on a real external provider, through a provider-adapter interface identical in shape to pinningProviders/* and legacyDataAccess/connectorRegistry.js.

COMPONENT	FILE	RESPONSIBILITY
Provider registry	rds/providerRegistry.js	Flat name -> module map, filtered by isConfigured() -- adding a second provider is one file, not an architecture change.
Supabase provider	rds/providers/supabase.js	Real calls to api.supabase.com/v1: create/list/get/pause/restore/delete a project, list backups, restore-PITR, run a SQL statement.
Control plane	rds/instances.js	Inaya's own record (name, status, department scoping), audit, Evidence Graph -- delegates every real operation to the provider, never talks to Supabase directly itself.
Credential handling	integrationCrypto.js (reused)	The database password is AES-256-GCM encrypted with the same key already used for OAuth tokens (INTEGRATION_ENCRYPTION_KEY) -- never stored or logged in plaintext.

Boundaries, stated plainly. Read replicas and failover are NOT claimed as supported -- Supabase's High Availability is a paid, project-level flag this provider can request, but failover behaviour has not been verified through a real incident, so createReplica()/failover() honestly return NOT_SUPPORTED rather than a guess.

SECTION 57

AI/ML Studio & Document Intelligence — Architecture (September 2026)

Document Intelligence generalizes bookkeeper/extract.js's proven deterministic-then-AI-corroborated pattern to an arbitrary analyzer field schema instead of a fixed invoice-field list. AI/ML Studio's governance layer is metadata over sources that already exist -- it never stores a second copy of any data.

COMPONENT	FILE	RESPONSIBILITY
Analyzer registry	docIntelligence/analyzers.js	6 built-in analyzers (synthesized in code, never stored) plus custom ones with a DRAFT->TESTING->READY->ACTIVE->DISABLED->ARCHIVED lifecycle.
Extraction engine	docIntelligence/extract.js	Schema-driven EXTRACT/CLASSIFY/GENERATE; reuses bookkeeper's textFromBuffer() and support/ai.js's gatedJson() (the existing AI Security Gateway) -- no second AI-calling seam.
Catalog	mlStudio/catalog.js	A TABLE/ANALYZER/DOCUMENT_PROJECT entry is VERIFIED against the real source (a legacyDataAccess virtual table, a Document Intelligence analyzer, or an org_documents project) before it is accepted.
Data quality	mlStudio/dataQuality.js	NOT_NULL/UNIQUE/RANGE/ROW_COUNT_MIN rules run through legacyDataAccess's EXISTING read-only SQL gateway -- no second query engine.
Model registry	mlStudio/models.js	Real artifact hashing (sha256) and encrypted, sharded storage (s3-compat/store.js, reused verbatim); same lifecycle taxonomy as analyzers.
Sandboxed execution	mlStudio/execution.js	One code snippet, one fresh @vercel/sandbox microVM, stopped immediately after -- network denied by default, Inaya's own server environment never passed in (env: {}), rate-limited, audited.

> **Why this is not a notebook, by design.** No saved kernel state between calls, no cell-to-cell variable sharing -- a persistent notebook needs a persistent sandbox and a session-management layer this pass does not build. Calling this a "notebook" would be exactly the unsafe pseudo-notebook the SOW that scoped this work says never to ship.

> Every new subject type (DOC_INTELLIGENCE_RESULT, RDS_INSTANCE, ML_STUDIO_MODEL) is registered in businessEvents.js's existing SUBJECT_RESOLVERS -- the same Evidence Graph every other feature uses, not a second one.

SECTION 58

Sovereign Infrastructure Fabric -- Core Architecture: VMs, Storage, Networking (October 2026)

A new sibling Rust workspace, inaya-infrastructure/, gives a customer's own connected hardware a real control plane: real QEMU/KVM virtual machines, real qcow2 block storage, real Linux-bridge+nftables networking, and a capacity-based scheduler -- authenticated node-to-node over signed requests on HTTPS (not literal TLS mTLS, a documented Vercel-edge platform constraint), surfaced through a new 'Infrastructure' console in the Business Workspace.

COMPONENT	FILE	RESPONSIBILITY
Node daemon entry point	inaya-infrastructure/crates/inaya-infra-node/src/main.rs	The long-running process a customer installs on their own hardware; registers with Inaya, polls for operations, reports capacity.
Operation executor	inaya-infrastructure/crates/inaya-infra-node/src/operations.rs	One match arm per operation type (CREATE_INSTANCE, BOOT_GUEST_INSTANCE, CREATE_VOLUME, CREATE_VPC, CREATE_LOAD_BALANCER, etc.) -- every new capability this round added is a new arm here, not a new executor.
Compute lifecycle	inaya-infrastructure/crates/compute/src/lib.rs	QemuKvmProvider: create/start/stop/restart/delete a real VM, tracked by PID, signalled on stop -- the same process-lifecycle pattern the load balancer crate reuses below.
Base image caching	inaya-infrastructure/crates/compute/src/cloud_image.rs	Downloads a real Ubuntu Server cloud image (qcow2) on first use, verifies it against the official published SHA256SUMS, caches it once per node.
Cloud-init seed	inaya-infrastructure/crates/compute/src/cloud_init.rs	Builds the NoCloud user-data/meta-data/network-config seed ISO a new guest boots with -- the piece that actually gets a VM online with a real static IP, not just an engine that starts and stops.
Block storage	inaya-infrastructure/crates/block/src/lib.rs	Real qcow2 volumes; create_volume_from_image() layers a fast copy-on-write overlay over the cached base image for new instances.
Networking	inaya-infrastructure/crates/network/src/lib.rs	LinuxBridgeNetworkProvider: one isolated bridge + nftables default-drop ruleset per customer VPC, plus a gateway IP, IP forwarding, and a scoped NAT/masquerade rule so guests reach the internet without reaching each other.
Guest SSH channel	inaya-infrastructure/crates/guest-exec/src/lib.rs	A pure-Rust SSH client (russh, no shelling out) that waits for guest connectivity with backoff and runs real commands/pulls real files inside a guest -- general-purpose, not k3s-specific; reused by the Kubernetes work in Section 59.
Node identity	inaya-infrastructure/crates/certificates/src/lib.rs	Per-node keypair issuance and the signed-request auth scheme every node-to-node call uses in place of literal mTLS.

COMPONENT	FILE	RESPONSIBILITY
Hardware telemetry	inaya-infrastructure/crates/telemetry/src/lib.rs	Heartbeat capacity reporting, extended this round with detect_tpm() and detect_confidential_compute_flags() -- detection only, same precedent as the existing GPU detection.
Terraform provider	terraform-provider-inaya (compute resources)	3 compute resources, intentionally separate from the pre-existing storage-only Terraform provider rather than merged into it -- different lifecycle, different blast radius.

A real, previously-missing prerequisite this round had to build. Before this round, no VM in this project had ever booted a real guest OS and come online -- every prior VM test proved the engine could start and stop, nothing more. Real guest boot (cloud_image.rs + cloud_init.rs + the network crate's NAT rule + guest-exec's SSH wait-with-backoff) had to be built from scratch, and is now reused by everything in Section 59.

A real, pre-existing bug this round found and fixed. NodeCapacity was serialized camelCase on one side of the wire and read snake_case on the other, so the scheduler's "pick the node with the most free memory" comparison silently never matched -- it never crashed or errored, it just quietly stopped picking the best node. Fixed with #[serde(rename_all = "camelCase")] in api-types, with its own regression test.

SECTION 59

Sovereign Infrastructure Fabric -- AWS CLI, Load Balancing & Kubernetes (Gap-Closure Round, October 2026)

The core architecture in Section 58 named three things deliberately left for later: AWS CLI/SDK compatibility, load balancing, and Kubernetes/container hosting. This gap-closure round built all three as a real, tested subset -- plus TPM/confidential-compute detection -- leaving only GPU passthrough out, for the same stated hardware reason as before.

COMPONENT	FILE	RESPONSIBILITY
SigV4 request signing	src/lib/infrastructure/awsSigV4.js	Real canonical-request construction and the real kDate->kRegion->kService->kSigning HMAC chain -- verified against AWS's own published SigV4 worked example, not a lookalike.
Access key issuance	src/lib/infrastructure/awsCredentials.js	Mints a real Inaya-issued key pair an organization uses with the actual aws CLI or boto3 via --endpoint-url, secret encrypted at rest with the same pattern nodeCerts.js already uses for CA keys.
EC2 query-protocol responses	src/lib/infrastructure/awsEc2Xml.js	Real EC2-shaped XML (botocore's EC2 model expects XML, not JSON) for RunInstances/DescribeInstances/Start/Stop/TerminateInstances and volume/VPC calls.
EC2 API entry point	src/app/api/aws-ec2/route.js	Single POST route dispatching on the real EC2 query protocol's Action form field, mapping each action straight onto the existing operations.js functions -- no second state model.
Load balancer process lifecycle	inaya-infrastructure/crates/loadbalancer/src/lib.rs	HaproxyLoadBalancerProvider: renders a real haproxy.cfg (TCP mode, round-robin, health checks) and spawns/reloads/stops the real haproxy process, reusing compute's own process-lifecycle pattern.
Load balancer records	src/lib/infrastructure/loadBalancers.js	CRUD over a new infra_load_balancers collection, mirroring operations.js's existing volume/VPC pattern -- required exporting enqueue() from operations.js, a zero-risk visibility change.
Kubernetes clusters	src/lib/infrastructure/k8sClusters.js	Tracks {k8sClusterId, serverInstanceId, agentInstanceIds[], kubeApiEndpoint}; retrieves the real k3s node-token and kubeconfig over the guest-exec SSH channel from Section 58, rewriting the server's loopback IP to its real static IP.

A real, pre-existing-pattern bug this round found and fixed. guest-exec's exec() broke its read loop on ChannelMsg::Eof, which worked by luck for short SSH responses (the node-token) but silently truncated longer ones (the kubeconfig) because ExitStatus can legitimately arrive after Eof for larger payloads. Fixed to only break on ChannelMsg::Close.

Proven for real, not simulated. A real Ubuntu cloud image boots under QEMU/KVM, gets its static IP from cloud-init, and reaches the public internet through the VPC's NAT rule. The real aws CLI genuinely creates and lists a real instance through this stack. A real haproxy process round-robins real TCP connections across two real backends, surviving a live config reload. A real k3s server and agent come up on real guest VMs, kubectl get nodes shows both Ready against the real downloaded kubeconfig, and a real test pod reaches Running. 82+ automated tests pass across both rounds (30 Rust, 52 Node.js).

What this deliberately is not. Not full AWS coverage -- S3, IAM and autoscaling are not included. Not HTTP-aware load balancing -- today's load balancer is TCP-only, no AWS-compatible load-balancer API. Not GPU passthrough -- no dedicated GPU host exists in this environment to test real PCI passthrough against, so it stays deferred rather than claimed. Not TPM-backed identity or real SEV/TDX execution -- hardware detection only, same honesty bar as the existing GPU-detection precedent.

SECTION 60

Secure Collaboration & Enterprise Governance — Architecture (October 2026)

A Filen/FileCloud-inspired suite built across 26 workstreams on top of the existing Business Workspace permission foundation -- end-to-end encrypted chat and notes, link sharing controls, an enterprise governance/compliance layer, device trust, and an on-premises gateway agent -- every feature off by default per organization.

COMPONENT	FILE	RESPONSIBILITY
MLS group messaging	src/lib/chat/mlsServer.js	Server-side MLS (RFC 9420) group state and message relay -- the server stores and forwards ciphertext, never holds a group's decryption-capable key.
Chat conversations & devices	src/lib/chat/{conversations,devices,contacts,presence}.js	Conversation membership, per-device MLS client registration, contact resolution and presence, all scoped to the existing org/department permission model.
Chat transport	src/lib/chat/transport.js	The same long-poll/SSE real-time pattern Meet's signaling layer reuses (Section 61) -- one real-time primitive for the whole platform, not a second one per feature.
Attachments	src/lib/chat/attachments.js	Encrypted file attachments over the existing sharded storage backend, not a second file-storage path.
Secure Notes	src/lib/notes/notes.js	End-to-end encrypted notes on the same client-side encryption model as chat, with the same device-key registry.
Sharing 2.0	src/lib/sharing/{shares,policy}.js	Expiring, password-protected and watermarked links plus inbound file requests, layered on the existing secure-sharing primitive rather than a second sharing engine.
Governance -- classification & DLP	src/lib/governance/{classification,classifyRules,dlp}.js	Document classification labels and data-loss-prevention rule matching, enforced at the point data would leave a scope.
Governance -- retention & lifecycle	src/lib/governance/{retention,lifecycle,metadata}.js	Automated retention/legal-hold policies and lifecycle transitions, journaled through governance/events.js for the audit trail.
Governance enforcement	src/lib/governance/enforce.js	The single checkpoint every governed action passes through -- policy evaluation lives here once, not duplicated per feature.

COMPONENT	FILE	RESPONSIBILITY
Sovereign Gateway	src/lib/gateway/{gateway,acl,transfers,health,modes}.js	An agent connecting a customer's own on-premises file server into Business Workspace, with its own ACL model, transfer queue, and health reporting -- built and tested but not yet proven against a real company file server.

Coverage, stated plainly. 114 tracked deliverables: 49 finished and independently checked against the real database and, where there's a screen, a real browser; 35 built and tested in code but not yet proven against a real outside system; 27 partly built; 3 not built by deliberate choice (mobile chat and push notifications). The Sovereign Gateway, the NTFS/Active-Directory folder-permission bridge, high-availability site replication, and Microsoft 365/Outlook integration fall in the "tested but not proven against a real outside system" group.

What this deliberately is not. The MLS protocol library's open-source implementation has not had an independent security audit. This stage stays explicitly in-progress -- the project's own final verification report says the SOW is not yet complete, and this document does not round that up.

SECTION 61

Post-Quantum Cryptography, Cleaner, Meet & Device Protection — Architecture (October 2026)

A hybrid PQC key-establishment layer that extends the existing sharing primitive rather than replacing it; a local-only device scanner that never leaves the desktop app's own process; peer-to-peer meeting media that never touches an Inaya server; and a reconciliation pass that added zero new backup engines after auditing eight existing ones.

COMPONENT	FILE	RESPONSIBILITY
PQC provider	custody-sdk/src/pqc/provider.js	ML-KEM-768 (hashes-only dependency) + X25519 combined through a domain-separated HKDF-SHA256 -- a hand-rolled combiner chosen specifically to avoid a real @noble/curves major-version conflict the package's own bundled hybrid preset would have forced onto every existing wallet-signing code path.
Algorithm-agile sharing	custody-sdk/src/pqc/agileSharing.js	LEGACY_CLASSICAL/HYBRID_PQC/PQC_REQUIRED modes over the SDK's existing X25519 sharing primitive -- the classical path is byte-for-byte the function that's always been there, never forked.
Device key registry	src/lib/pqc/deviceKeys.js	Public keys only, scoped to an already-checked-in device; revocation now wired into the existing device-revocation workflow rather than left as a standalone, unused function.
Cleaner	inaya-desktop/src-tauri/src/cleaner.rs	Local filesystem scan only -- no file inventory ever leaves the device. Protected paths re-checked at the actual deletion call, not just at scan time; OS recycle bin, never a permanent delete, by default.
Meet room lifecycle + signaling	src/lib/meet/{rooms,signaling}.js	Signaling over the same long-poll/SSE pattern Secure Chat's own real-time layer already uses -- the honest architectural answer on a serverless platform that cannot hold a persistent connection open. Media is mesh peer-to-peer; no SFU exists in this version, a deliberate v1 scope decision, not an oversight (docs/architecture/meet-architecture-adr.md).
Meet client	src/components/business/meet/MeetView.js	Real RTCPeerConnection negotiation and client-side PQC key wrapping for admission -- the room secret is generated on the host's own device and never reaches the server in plaintext.
Device Protection	src/components/business/DeviceProtectionView.js	Composition only -- Backup, Sync, Cleaner and Device status tabs each render an existing, unmodified view. No logic moved, no new data path.

Why backup reconciliation added no new engine. An audit of all 8 existing backup-related files found nearly every behavior the SOW asked about already real -- scheduling, pause/resume, retry queues, point-in-time restore, and a ransomware-aware recovery flow requiring a second administrator's approval were all already there. The only genuinely new work was UI consolidation and a Cleaner-aware warning before deleting a file inside an active backup folder.

A live pass against a real running server and a real browser found and fixed two genuine bugs before this was written up -- a pre-existing crash unrelated to this work, and a Meet-specific false error for a user with no camera. A real two-person video call and a real mobile device run remain the two pieces needing real-world hardware this pass could not provide.